

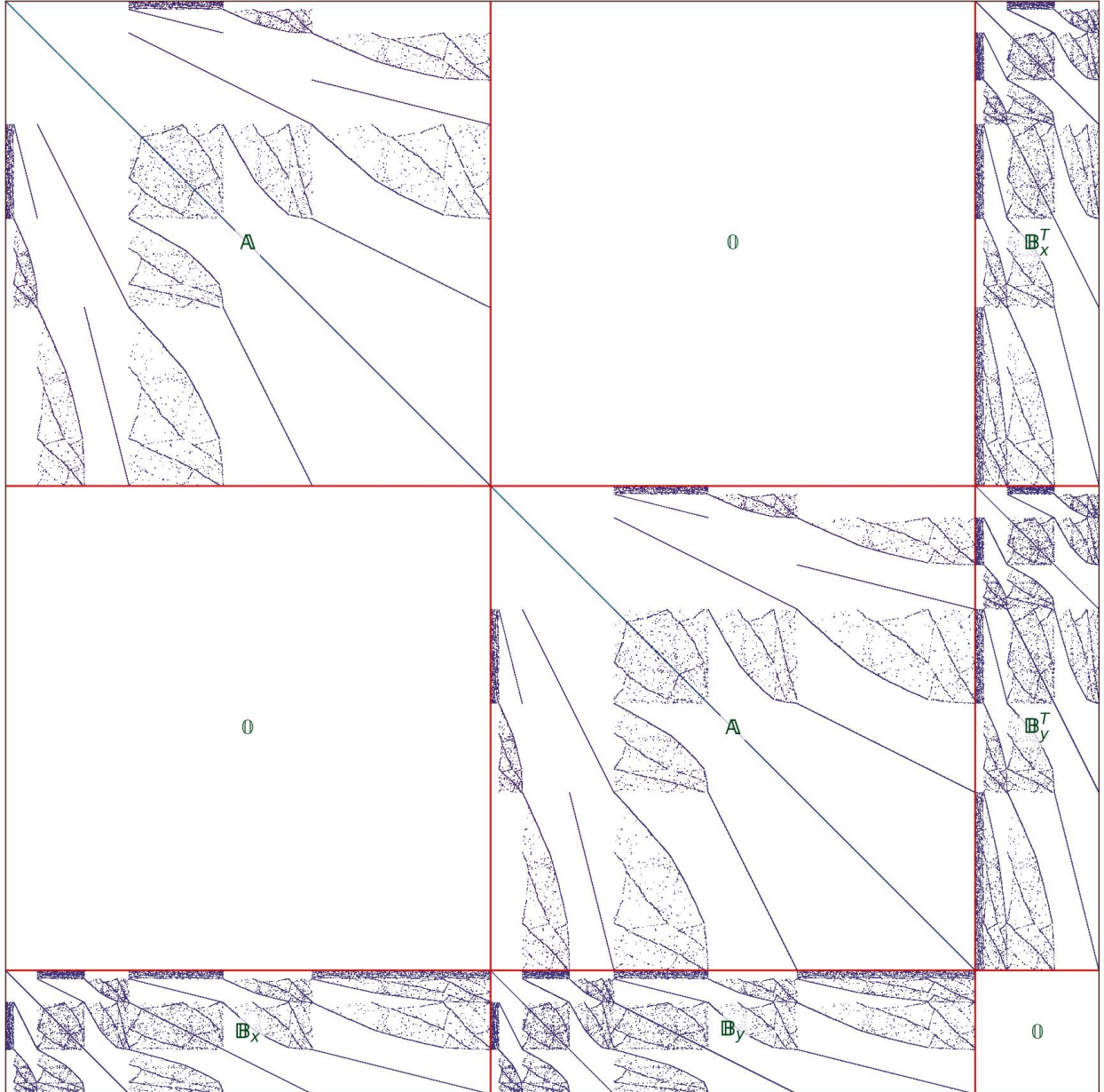
Finite Element and Reduced-Order Modeling of Parameterized Stokes Flows

Oleksandr Samoliuk

Summer Semester, 2026

Abstract

This thesis presents a finite element framework for the numerical simulation of parameterized incompressible Stokes flows together with a reduced-order modeling approach for efficient many-query computations. A stable mixed finite element discretization is employed to accurately resolve the coupled velocity and pressure fields while satisfying the incompressibility constraint. Based on the resulting full-order model, a reduced-order model is constructed using Proper Orthogonal Decomposition, enabling efficient approximation of the solution over varying parameter values. The proposed methodology is validated through numerical experiments, demonstrating that the reduced model achieves substantial computational savings while maintaining high accuracy with respect to the full-order simulations.



Contents

1	Introduction to Stokes Equations and Finite Element Method	3
1.1	Mathematical Characterization and Challenges	3
1.2	Stable Macro-Element Discretizations: The P_1 -iso- P_1 Element	3
2	Continuous and Variational Formulations	4
3	Finite Element Discretization and Matrix Assembly	5
3.1	Reference Triangle	5
3.2	Stiffness Matrix $\mathbf{A}$	6
3.3	Pressure Matrices $\mathbf{B}_x$ and $\mathbf{B}_y$ on the P_1 -iso- P_1 Macro-Element	7
3.4	Saddle-Point Block Matrix $\mathbf{K}$	8
3.5	Load Vector $\mathbf{F}$ Assembly and Inhomogeneous Boundary Lifting	8
3.6	Pressure Null-Space Regulation and Singular Constraints	9
4	Ladyzhenskaya–Babuška–Brezzi Condition	10
4.1	Discrete Mapping and Coordinate Representation	10
4.2	Discretization Proofs for Operators and Field Norms	10
4.3	The Final Discrete Matrix Form	12
4.4	Solving the Optimization Problem	12
4.5	Numerical Stability Assessment	13
5	Numerical Experiments and Applications	15
6	Convergence of Numerical Solutions	17
7	Introduction to Reduced-Order Modeling	19
8	Parametrized Problem Formulation	19
8.1	The Viscosity Parameter Vector $\boldsymbol{\mu}$	19
8.2	Affine Decomposition of the Viscous Operator	20
9	Data Generation	21
9.1	Full-Order Snapshot Collection	21
9.2	Supremizer Enrichment for Inf-Sup Stability	21
9.3	Velocity Snapshot Homogenization via Boundary Lifting	23
10	Reduced Basis Construction	23
10.1	Proper Orthogonal Decomposition and Energy Truncation	23
10.2	Reduced Velocity and Pressure Spaces	23
11	Offline–Online Decomposition	23
11.1	Offline Phase: Reduced Operator Assembly	23
11.2	Online Phase: The Reduced Galerkin System	23
12	Numerical Results	24
12.1	Train/Test Partitioning of the Parameter Space	24
12.2	Reconstruction Accuracy: Seen vs. Unseen Parameters	24
12.3	Velocity and Pressure Field Comparison (FOM vs. ROM)	24
12.4	Derived Quantities of Interest	24
12.5	Computational Speedup	24
13	Appendix	25
13.1	Reverse Techniques: Einsum	25

1 Introduction to Stokes Equations and Finite Element Method

Stokes flow, or *creeping flow*, describes incompressible fluid motion in which viscous forces dominate inertial effects, characterized by a low Reynolds number,

$$\text{Re} = \frac{\rho U L}{\mu} \ll 1, \quad \nu = \frac{\mu}{\rho},$$

where ρ is the fluid density, U a characteristic velocity, L a characteristic length scale, μ the dynamic viscosity, and ν the kinematic viscosity. The regime arises whenever motion is slow, viscosity is high, or the length scale is very small, making it the governing model for microfluidics, MEMS, coating and polymer processing, biological transport, microorganism locomotion, particle settling, and slow mantle convection. First analyzed by Sir George Gabriel Stokes in his study of viscous settling, the equations remain today a foundational model in fluid mechanics and the basis for a wide range of numerical and reduced-order methods.

1.1 Mathematical Characterization and Challenges

The Stokes equations follow from linearizing the incompressible Navier–Stokes equations,

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} \\ \nabla \cdot \mathbf{u} = 0 \end{cases}$$

where $\mathbf{u}$ is the velocity field, t time, p the pressure field, ρ the constant density, ν the kinematic viscosity, and $\mathbf{f}$ external body forces. Dropping the convective term $(\mathbf{u} \cdot \nabla) \mathbf{u}$ linearizes the momentum equation, leaving $\mathbf{u}$ and p coupled purely through viscous dissipation and incompressibility.

This simplification nonetheless introduces two structural challenges for the Finite Element Method:

- **Saddle-Point Structure:** The constraint $\nabla \cdot \mathbf{u} = 0$ gives the weak formulation a *saddle-point* (constrained-optimization) algebraic structure.
- **The Inf-Sup (LBB) Condition:** Velocity and pressure spaces X_h, Q_h cannot be paired arbitrarily as in standard elliptic problems; unstable pairings produce spurious “checkerboard” pressure oscillations. Stability requires the *Ladyzhenskaya-Babuška-Brezzi (LBB)* condition, satisfied by pairings such as Taylor-Hood (P_k/P_{k-1}) or stabilized formulations.

1.2 Stable Macro-Element Discretizations: The P_1 -iso- P_1 Element

The P_1 -iso- P_1 element, introduced by Bercovier and Pironneau, satisfies the LBB condition without resorting to higher-order or non-conforming spaces. Rather than varying polynomial degree, it varies mesh density: both velocity and pressure remain piecewise-linear (P_1), but velocity lives on a fine mesh $\mathcal{T}_h$ obtained by uniformly refining a coarse macro-mesh $\mathcal{T}_H$ – each macro-triangle $K \in \mathcal{T}_H$ split into four congruent children by connecting edge midpoints, halving the mesh size to $h = H/2$.

The discrete spaces are

$$\begin{aligned} X_h &= \{\mathbf{v}_h \in C^0(\bar{\Omega})^2 : \mathbf{v}_h|_K \in P_1(K)^2, \forall K \in \mathcal{T}_h\} \\ Q_h &= \{q_h \in C^0(\bar{\Omega}) : q_h|_M \in P_1(M), \forall M \in \mathcal{T}_H\} \end{aligned}$$

Placing $\mathbf{u}_h$ on the fine grid and p_h on the coarse grid enriches velocity with far more internal degrees of freedom than pressure, yielding a stable ratio that satisfies the discrete LBB condition,

$$\exists \beta > 0 : \inf_{q \in Q \setminus \{0\}} \sup_{v \in V \setminus \{0\}} \left(\frac{\int_{\Omega} q(\nabla \cdot v) d\Omega}{\|v\|_V \cdot \|q\|_Q} \right) \geq \beta$$

with β bounded away from zero independently of refinement. The result is a highly symmetric macro-element that keeps the low cost of linear shape functions while retaining robust convergence.

2 Continuous and Variational Formulations

Strong Formulation

The steady-state motion of an incompressible viscous fluid in the low-Reynolds-number regime is governed by the classical *Stokes equations*. These equations constitute a fundamental linear model in fluid mechanics and describe the balance between viscous stresses, pressure forces, and the incompressibility constraint. Given a bounded domain $\Omega \subset \mathbb{R}^2$ enclosed by a boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$ (where $\Gamma_D \cap \Gamma_N = \emptyset$), the strong formulation of the problem seeks the velocity field $\mathbf{u}(x, y)$ and the scalar pressure field $p(x, y)$ satisfying:

$$\begin{cases} -\nu\Delta\mathbf{u} + \nabla p = \mathbf{0} & \text{in } \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{on } \Gamma_D, \\ -p\mathbf{n} + \nu(\nabla\mathbf{u})\mathbf{n} = \mathbf{h} & \text{on } \Gamma_N. \end{cases}$$

The mathematical and physical roles of these individual components are summarized as follows:

- **Momentum Balance** ($-\nu\Delta\mathbf{u} + \nabla p = \mathbf{0}$): This represents the conservation of momentum. It dictates a strict equilibrium between the internal viscous shear stresses—scaled by the kinematic viscosity ν —and the driving force generated by the pressure gradient ∇p .
- **Incompressibility Constraint** ($\nabla \cdot \mathbf{u} = 0$): This serves as the conservation of mass equation for an incompressible fluid. It requires the velocity field to be divergence-free, ensuring that fluid volume is perfectly preserved at every point within the domain.
- **Dirichlet Boundary Condition** ($\mathbf{u} = \mathbf{g}$): Enforced on the boundary segment Γ_D , this prescribes the exact fluid velocity. It is commonly used to dictate non-zero flow profiles at fluid inlets or to impose the physical “no-slip” condition ($\mathbf{u} = \mathbf{0}$) along stationary solid walls.
- **Neumann Boundary Condition** ($-p\mathbf{n} + \nu(\nabla\mathbf{u})\mathbf{n} = \mathbf{h}$): Enforced on the boundary segment Γ_N with outward unit normal vector $\mathbf{n}$, this specifies the weak normal stress or traction force vector $\mathbf{h}$ acting on the fluid, typically representing free-pressure outlets or open boundaries.

Weak Formulation

To solve the Stokes system using the Finite Element Method, we transform the original strong partial differential equations into their weak (or variational) integral form. This is achieved by multiplying the momentum and continuity equations by smooth test functions, $\mathbf{v}$ and q , and integrating the terms over the entire physical domain Ω . Applying integration by parts (Green’s identity) to the second-order viscous diffusion term reduces the continuity requirements on our velocity fields from second-order derivatives down to first-order derivatives.

During this mathematical transition, the natural boundary conditions—such as the traction force $\mathbf{h}$ on the Neumann boundary Γ_N —automatically emerge within the boundary integrals. To accommodate non-zero (inhomogeneous) Dirichlet velocity profiles on the walls while strictly maintaining a homogeneous test function space X where $\mathbf{v} = \mathbf{0}$ on Γ_D , we apply a boundary lifting technique. We decompose the true velocity field into $\mathbf{u} = \mathbf{u}_h + \mathbf{r}_g$, where $\mathbf{r}_g$ is a known extension function matching the boundary data, and $\mathbf{u}_h \in X$ is the remaining homogeneous unknown field. This systematic process yields the coupled weak formulation system:

$$\begin{cases} \int_{\Omega} \nu \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega - \int_{\Omega} p (\nabla \cdot \mathbf{v}) \, d\Omega = \int_{\Gamma_N} \mathbf{h} \cdot \mathbf{v} \, d\Gamma - \int_{\Omega} \nu \nabla \mathbf{r}_g : \nabla \mathbf{v} \, d\Omega, & \forall \mathbf{v} \in X, \\ - \int_{\Omega} q (\nabla \cdot \mathbf{u}) \, d\Omega = \int_{\Omega} q (\nabla \cdot \mathbf{r}_g) \, d\Omega, & \forall q \in Q. \end{cases} \quad (1)$$

3 Finite Element Discretization and Matrix Assembly

In this section, we transition from the continuous **weak formulation** to a **discrete algebraic representation** suitable for computational execution. By projecting the infinite-dimensional solution space onto a finite-dimensional subspace spanned by piecewise linear basis functions, we transform the governing integral operators into global sparse matrices. The following subsections detail the systematic assembly of these components.

3.1 Reference Triangle

To compute integrals efficiently, we map every arbitrary triangle in our mesh onto a standard master element called the reference triangle $\hat{T}$ in local coordinates (ξ_1, ξ_2) . Its vertices are fixed at $\hat{\mathbf{x}}_1 = (0, 0)$, $\hat{\mathbf{x}}_2 = (1, 0)$, and $\hat{\mathbf{x}}_3 = (0, 1)$. On this master domain, we define the standard linear Lagrange shape functions $\hat{\phi}_i(\xi_1, \xi_2)$:

$$\begin{aligned} \hat{\phi}_1(\xi_1, \xi_2) &= 1 - \xi_1 - \xi_2 & \hat{\phi}_1(0, 0) &= 1, \quad \hat{\phi}_1(1, 0) = 0, \quad \hat{\phi}_1(0, 1) = 0 \\ \hat{\phi}_2(\xi_1, \xi_2) &= \xi_1 & \hat{\phi}_2(0, 0) &= 0, \quad \hat{\phi}_2(1, 0) = 1, \quad \hat{\phi}_2(0, 1) = 0 \\ \hat{\phi}_3(\xi_1, \xi_2) &= \xi_2 & \hat{\phi}_3(0, 0) &= 0, \quad \hat{\phi}_3(1, 0) = 0, \quad \hat{\phi}_3(0, 1) = 1 \end{aligned}$$

We will also take the gradients for easier future derivations and implementations:

$$\nabla \hat{\phi}_1(\xi_1, \xi_2) = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \quad \nabla \hat{\phi}_2(\xi_1, \xi_2) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \nabla \hat{\phi}_3(\xi_1, \xi_2) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

We will now take a close look at the mapping and derive a Jacobian:

$$x(\xi_1, \xi_2) = x_1 + (x_2 - x_1)\xi_1 + (x_3 - x_1)\xi_2$$

$$y(\xi_1, \xi_2) = y_1 + (y_2 - y_1)\xi_1 + (y_3 - y_1)\xi_2$$

The mapping can be easily checked:

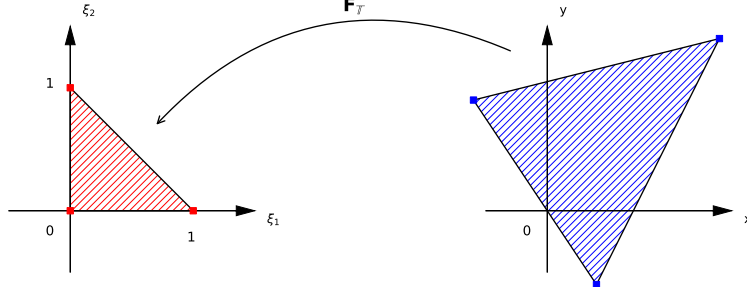
$$\begin{aligned} x(0, 0) &= x_1 & x(1, 0) &= x_2 & x(0, 1) &= x_3 \\ y(0, 0) &= y_1 & y(1, 0) &= y_2 & y(0, 1) &= y_3 \end{aligned}$$

We can now take the Jacobian:

$$\mathbf{J}_T = \begin{bmatrix} \frac{\partial x}{\partial \xi_1} & \frac{\partial x}{\partial \xi_2} \\ \frac{\partial y}{\partial \xi_1} & \frac{\partial y}{\partial \xi_2} \end{bmatrix} = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}$$

It is also necessary to take a look at the original shape functions' derivatives:

$$\begin{aligned} \nabla \phi_i(x, y) &= \begin{bmatrix} \frac{\partial \phi_i}{\partial x} \\ \frac{\partial \phi_i}{\partial y} \end{bmatrix} = \mathbf{J}_T^{-T} \cdot \nabla \hat{\phi}_i(\xi_1, \xi_2) = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix}^{-T} \cdot \begin{bmatrix} \frac{\partial \hat{\phi}_i}{\partial \xi_1} \\ \frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix} = \\ &= \left| \mathbf{X} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \right|, \quad \mathbf{X}^{-1} = \frac{1}{ad - cb} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \Big| = \frac{1}{|\mathbf{J}_T|} \begin{bmatrix} y_3 - y_1 & y_1 - y_2 \\ x_1 - x_3 & x_2 - x_1 \end{bmatrix} \cdot \begin{bmatrix} \frac{\partial \hat{\phi}_i}{\partial \xi_1} \\ \frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix} = \quad (2) \\ &= |\mathbf{J}_T|^{-1} \begin{bmatrix} (y_3 - y_1) \frac{\partial \hat{\phi}_i}{\partial \xi_1} - (y_2 - y_1) \frac{\partial \hat{\phi}_i}{\partial \xi_2} \\ -(x_3 - x_1) \frac{\partial \hat{\phi}_i}{\partial \xi_1} + (x_2 - x_1) \frac{\partial \hat{\phi}_i}{\partial \xi_2} \end{bmatrix} \end{aligned}$$



3.2 Stiffness Matrix A

To determine the discrete operator for the viscous term, we begin with the bilinear form involving the velocity test and trial functions:

$$\int_{\Omega} \nu \nabla u : \nabla v \, d\Omega$$

$$\begin{aligned} \int_{\Omega} \nu \nabla \mathbf{u}(x, y) : \nabla \mathbf{v}(x, y) \, d\Omega &= \left| \begin{array}{l} \mathbf{u}(x, y) = \begin{bmatrix} u_x(x, y) \\ u_y(x, y) \end{bmatrix} \\ \mathbf{v}(x, y) = \begin{bmatrix} v_x(x, y) \\ v_y(x, y) \end{bmatrix} \end{array} \right| = \int_{\Omega} \nu (\nabla u_x \cdot \nabla v_x + \nabla u_y \cdot \nabla v_y) \, d\Omega = \\ &= \int_{\Omega} \nu \nabla u_x \cdot \nabla v_x \, d\Omega + \int_{\Omega} \nu \nabla u_y \cdot \nabla v_y \, d\Omega = \left| \begin{array}{l} u_x(x, y) \approx \sum_{i=1}^{N_v} u_i^x \phi_i(x, y) \\ u_y(x, y) \approx \sum_{i=1}^{N_v} u_i^y \phi_i(x, y) \end{array} \right| = \\ &= \int_{\Omega} \nu \nabla \left(\sum_{i=1}^{N_v} u_i^x \phi_i \right) \cdot \nabla v_x \, d\Omega + \int_{\Omega} \nu \nabla \left(\sum_{i=1}^{N_v} u_i^y \phi_i \right) \cdot \nabla v_y \, d\Omega = \\ &= \left[\sum_{i=1}^{N_v} u_i^x \cdot \int_{\Omega} \nu \nabla \phi_i \cdot \nabla v_x \, d\Omega \right] + \left[\sum_{i=1}^{N_v} u_i^y \cdot \int_{\Omega} \nu \nabla \phi_i \cdot \nabla v_y \, d\Omega \right] = \\ &= \left| \begin{array}{l} \text{We now set } v_x(x, y) = \phi_j(x, y) \\ \text{and } v_y(x, y) = \phi_j(x, y) \end{array} \right| = \\ &= \left[\sum_{i=1}^{N_v} u_i^x \cdot \int_{\Omega} \nu \nabla \phi_i \cdot \nabla \phi_j \, d\Omega \right] + \left[\sum_{i=1}^{N_v} u_i^y \cdot \int_{\Omega} \nu \nabla \phi_i \cdot \nabla \phi_j \, d\Omega \right] = \\ &= \sum_{i=1}^{N_v} A_{ji} u_i^x + \sum_{i=1}^{N_v} A_{ji} u_i^y = \mathbf{A}^T \mathbf{u}_x + \mathbf{A}^T \mathbf{u}_y = |\mathbf{A} = \mathbf{A}^T| = \\ &= \mathbf{A} \mathbf{u}_x + \mathbf{A} \mathbf{u}_y = \begin{bmatrix} \mathbf{A} & \mathbf{A} \end{bmatrix} \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \end{bmatrix} \end{aligned}$$

Now using the derived coordinate transformation, performed with the help of the Jacobian, we can

calculate the local matrix $\mathbf{A}^{loc} \in \mathbb{R}^{3 \times 3}$

$$\mathbf{A}_{ij} = \int_{\Omega} \nu \nabla \phi_i(x, y) \cdot \nabla \phi_j(x, y) d\Omega$$

$$\begin{aligned} \mathbf{A}_{ij}^{loc} &= \left| \begin{array}{l} \nabla \phi(x, y) = \mathbf{J}_T^{-T} \nabla \hat{\phi}(\xi_1, \xi_2) \\ d\Omega = dx dy = |\mathbf{J}_T| d\xi_1 d\xi_2 \end{array} \right| = \\ &= \int_{\hat{T}} \nu \left(\mathbf{J}_T^{-T} \nabla \hat{\phi}_i(\xi_1, \xi_2) \right)^T \left(\mathbf{J}_T^{-T} \nabla \hat{\phi}_j(\xi_1, \xi_2) \right) |\mathbf{J}_T| d\xi_1 d\xi_2 = \int_{\hat{T}} \nu \left(\nabla \hat{\phi}_i \right)^T \mathbf{J}_T^{-1} \mathbf{J}_T^{-T} \left(\nabla \hat{\phi}_j \right) |\mathbf{J}_T| d\xi_1 d\xi_2 = \\ &= \int_{\hat{T}} \frac{\nu}{2 |\mathbf{J}_T|} \left(\nabla \hat{\phi}_i \right)^T \begin{bmatrix} y_3 - y_1 & -(x_3 - x_1) \\ -(y_2 - y_1) & x_2 - x_1 \end{bmatrix} \begin{bmatrix} y_3 - y_1 & -(y_2 - y_1) \\ -(x_3 - x_1) & x_2 - x_1 \end{bmatrix} \left(\nabla \hat{\phi}_j \right) |\mathbf{J}_T| d\xi_1 d\xi_2 = \\ &= \frac{\nu}{|\mathbf{J}_T|} \int_{\hat{T}} \left(\nabla \hat{\phi}_i \right)^T \mathbf{Q} \left(\nabla \hat{\phi}_j \right) d\xi_1 d\xi_2 = \frac{\nu}{2 |\mathbf{J}_T|} \left(\nabla \hat{\phi}_i \right)^T \mathbf{Q} \left(\nabla \hat{\phi}_j \right) \end{aligned}$$

After calculating the vector-matrix-vector products explicitly using the constant reference gradients $\nabla \hat{\phi}_1 = [-1, -1]^T$, $\nabla \hat{\phi}_2 = [1, 0]^T$, and $\nabla \hat{\phi}_3 = [0, 1]^T$, we obtain the fully expanded local element matrix:

$$\mathbf{A}^{loc} = \frac{\nu}{|\mathbf{J}_T|} \begin{bmatrix} \nabla \hat{\phi}_1 \mathbf{Q} \nabla \hat{\phi}_1 & \nabla \hat{\phi}_1 \mathbf{Q} \nabla \hat{\phi}_2 & \nabla \hat{\phi}_1 \mathbf{Q} \nabla \hat{\phi}_3 \\ \nabla \hat{\phi}_2 \mathbf{Q} \nabla \hat{\phi}_1 & \nabla \hat{\phi}_2 \mathbf{Q} \nabla \hat{\phi}_2 & \nabla \hat{\phi}_2 \mathbf{Q} \nabla \hat{\phi}_3 \\ \nabla \hat{\phi}_3 \mathbf{Q} \nabla \hat{\phi}_1 & \nabla \hat{\phi}_3 \mathbf{Q} \nabla \hat{\phi}_2 & \nabla \hat{\phi}_3 \mathbf{Q} \nabla \hat{\phi}_3 \end{bmatrix} \in \mathbb{R}^{3 \times 3}, \quad \text{where:}$$

$$\mathbf{Q} = \mathbf{J}_T^{-1} \mathbf{J}_T^{-T} = \begin{bmatrix} (y_3 - y_1)^2 + (x_3 - x_1)^2 & (y_1 - y_2)(y_3 - y_1) + (x_2 - x_1)(x_1 - x_3) \\ (y_1 - y_2)(y_3 - y_1) + (x_2 - x_1)(x_1 - x_3) & (y_2 - y_1)^2 + (x_2 - x_1)^2 \end{bmatrix}$$

Please note that the matrices $\mathbf{Q}$ and $\mathbf{A}^{loc}$ are symmetric. This structural property is highly advantageous for optimizing numerical calculations.

3.3 Pressure Matrices $\mathbf{B}_x$ and $\mathbf{B}_y$ on the P_1 -iso- P_1 Macro-Element

Unlike a conforming equal-order pairing, the pressure basis function ψ_j is supported on the *macro* triangle $T^c \in \mathcal{T}_H$, while the velocity basis functions ϕ_i are supported on the four *child* triangles $T_1^f, T_2^f, T_3^f, T_4^f \in \mathcal{T}_h$ obtained by connecting the edge midpoints of T^c . Consequently the coupling integral must be split into a sum over the four children, and the pressure basis function must be re-expressed in each child's own local reference coordinates. This restriction is the main idea of the P_1 -iso- P_1 assembly.

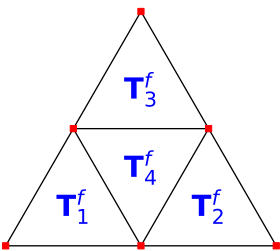
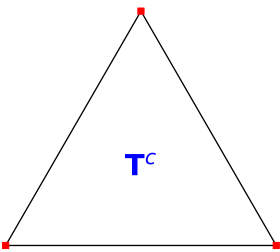


Figure 1: Macro element and its corresponding subdivision into 4 congruent elements.

Weak formulation

$$-\int_{\Omega} p(\nabla \cdot \mathbf{v}) d\Omega$$

3.4 Saddle-Point Block Matrix $\mathbf{K}$

By organizing the viscous diffusion blocks and the divergence constraint blocks into a single monolithic operator, we construct the global saddle-point system. This specific structural arrangement takes the classic Karush-Kuhn-Tucker (KKT) form:

$$\begin{bmatrix} \mathbf{A} & 0 & \mathbf{B}_x^T \\ 0 & \mathbf{A} & \mathbf{B}_y^T \\ \mathbf{B}_x & \mathbf{B}_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \\ \mathbf{0} \end{bmatrix}$$

The block layout reflects the underlying **mixed formulation** (1), where the diagonal velocity stiffness blocks $\mathbf{A}$ remain symmetric positive definite, while the zero block on the lower diagonal highlights the lack of an explicit pressure time-dependence or compressibility stabilizer. The dimensional layout of these individual blocks scales according to the total degrees of freedom allocated to each field variable:

$$\mathbf{A} \in \mathbb{R}^{N_u \times N_u} \quad \mathbf{B}_x \in \mathbb{R}^{N_p \times N_u} \quad \mathbf{B}_x^T \in \mathbb{R}^{N_u \times N_p}$$

3.5 Load Vector $\mathbf{F}$ Assembly and Inhomogeneous Boundary Lifting

$$\mathbf{F} = \int_{\Gamma_N} h \cdot \mathbf{v} d\Gamma - \int_{\Omega} \nu \nabla \mathbf{r}_g : \nabla \mathbf{v} d\Omega = \mathbf{F}_{forces} - \mathbf{A} \vec{r}_g = \begin{bmatrix} F_x \\ F_y \end{bmatrix} - \mathbf{A} \vec{r}_g$$

Construction of the Lifting Function $\mathbf{r}_g$

The strong Dirichlet condition reads $\mathbf{u} = \mathbf{g}$ on Γ_D , and the weak formulation is stated for the homogeneous unknown $\mathbf{u} \in X$ via the decomposition $\mathbf{u} = \mathbf{u}_h + \mathbf{r}_g$. Discretely, $\mathbf{r}_g \in \mathbb{R}^{2N_v}$ is simply the nodal interpolant of $\mathbf{g}$. For every fine-mesh node X_i ,

$$(\mathbf{r}_g^x)_i = \begin{cases} g_x(X_i) & X_i \in \Gamma_D^{\text{inlet}}, \\ 0 & \text{otherwise,} \end{cases} \quad (\mathbf{r}_g^y)_i = 0 \quad \forall i, \quad (3)$$

where $\Gamma_D = \Gamma_D^{\text{inlet}} \cup \Gamma_D^{\text{wall}}$ collects every boundary node except the outlet which is left as the natural, traction-free boundary Γ_N , so $h = \mathbf{0}$ and $\mathbf{F}_{forces} = \mathbf{0}$ in the later experiments. Walls carry the homogeneous no-slip value $g = \mathbf{0}$ automatically, since $\mathbf{r}_g$ is zero everywhere it is not explicitly prescribed.

Momentum and Continuity Right-Hand Sides

With $\mathbf{F}_{forces} = \mathbf{0}$, the momentum block reduces directly to $\mathbf{F} = -\mathbf{A} \vec{r}_g$ as stated above. The continuity equation of the weak system (1) needs the same treatment: testing $-\int_{\Omega} q(\nabla \cdot \mathbf{u}) d\Omega = \int_{\Omega} q(\nabla \cdot \mathbf{r}_g) d\Omega$ against the discrete pressure basis and applying the identity of §5.2.1 to *both* sides (once for the homogeneous unknown $\mathbf{u}$, once for the known lift $\vec{r}_g$) gives

$$-\mathbf{p}^T \mathbf{B} \mathbf{u} = \mathbf{p}^T \mathbf{B} \vec{r}_g \quad \forall \mathbf{p} \implies \mathbf{B}_x \mathbf{u}_x + \mathbf{B}_y \mathbf{u}_y = -(\mathbf{B}_x \vec{r}_g^x + \mathbf{B}_y \vec{r}_g^y).$$

Assembling all three rows, the full right-hand side of the lifted saddle-point system is

$$\begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} -\mathbf{A} \vec{r}_g^x \\ -\mathbf{A} \vec{r}_g^y \\ -(\mathbf{B}_x \vec{r}_g^x + \mathbf{B}_y \vec{r}_g^y) \end{bmatrix}.$$

Enforcing the Homogeneous Dirichlet Condition

Because $\mathbf{u} \in X$ requires $\mathbf{u} = \mathbf{0}$ exactly on Γ_D , every degree of freedom $i \in \Gamma_D$ must be pinned to zero in the linear system itself, not merely approximated. This is done by row substitution: for each such i , row i of $\mathbf{K}$ is replaced by the i -th unit row and F_i is set to zero,

$$\mathbf{K}_{i,:} \leftarrow \mathbf{e}_i^T, \quad F_i \leftarrow 0,$$

which forces $u_i = 0$ directly upon solving. Crucially, only the *row* is overwritten — the column $\mathbf{K}_{:,i}$ is left untouched. This is exact precisely because the imposed value is zero: for every other equation j , the term $\mathbf{K}_{j,i} u_i = \mathbf{K}_{j,i} \cdot 0 = 0$ automatically, so no compensating correction to F_j is required. This is the payoff of lifting in the first place — had the original inhomogeneous problem been assembled directly for $\mathbf{u}_{total}$, enforcing $u_{total,i} = g_i \neq 0$ this way would require subtracting $\mathbf{K}_{j,i} g_i$ from every other row's right-hand side; working with the homogeneous unknown $\mathbf{u}$ instead reduces Dirichlet enforcement to the cheapest possible operation.

3.6 Pressure Null-Space Regulation and Singular Constraints

Pressure enters the momentum equation only through the coupling matrices $\mathbf{B}_x^T, \mathbf{B}_y^T$, and in this formulation the outlet is left as a natural, do-nothing boundary rather than one carrying an explicit pressure reference. As a result, nothing in the assembled system fixes an absolute pressure level: the discrete saddle-point matrix $\mathbf{K}$ is singular (or, numerically, severely ill-conditioned) in its pressure block, exactly as in a fully enclosed cavity flow, and a direct sparse solve of $\mathbf{K}\vec{s} = \mathbf{F}$ is not well-posed as stated.

The remedy is the same row-substitution technique used for the velocity Dirichlet nodes, applied to a single pressure degree of freedom p_{ref} chosen at (or nearest to) the outlet:

$$\mathbf{K}_{p_{\text{ref}},:} \leftarrow \mathbf{e}_{p_{\text{ref}}}^T, \quad F_{p_{\text{ref}}} \leftarrow 0,$$

which fixes $p_{p_{\text{ref}}} = 0$ and removes the indeterminacy, leaving every other pressure value defined relative to this reference. As before, only the row is overwritten: the column $\mathbf{K}_{:,p_{\text{ref}}}$ is preserved so that the (now fixed, zero) reference pressure still correctly participates in every other momentum equation, by the same zero-contribution argument as above. If the mesh has no coarse node lying exactly on the outlet boundary, the reference is instead taken as the coarse pressure node nearest to the outlet's midpoint, ensuring the regularization is always well-defined regardless of mesh irregularity.

4 Ladyzhenskaya–Babuška–Brezzi Condition

In numerical partial differential equations, the *Ladyzhenskaya–Babuška–Brezzi* (LBB) condition is a sufficient condition for a saddle point problem to have a unique solution that depends continuously on the input data.

At the continuous level, the velocity space V and pressure space Q must satisfy:

$$\exists \beta > 0 : \quad \inf_{q \in Q \setminus \{0\}} \sup_{v \in V \setminus \{0\}} \left(\frac{\int_{\Omega} q(\nabla \cdot v) d\Omega}{\|v\|_V \cdot \|q\|_Q} \right) \geq \beta$$

4.1 Discrete Mapping and Coordinate Representation

For this problem it is useful to reformulate the condition in a discrete way. We can start by mapping the continuous functions to their nodal coordinate representations:

$$v \rightarrow \mathbf{u} = \begin{bmatrix} u_1^x \\ \vdots \\ u_{N_v}^x \\ u_1^y \\ \vdots \\ u_{N_v}^y \end{bmatrix} \in \mathbb{R}^{2N_v}, \quad q \rightarrow \mathbf{p} = \begin{bmatrix} p_1 \\ \vdots \\ p_{N_p} \end{bmatrix} \in \mathbb{R}^{N_p}$$

The corresponding discrete differential bilinear forms translate directly into the global block matrices assembled across the mesh:

$$\begin{aligned} \mathbf{A}_{ij} &= \int_{\Omega} \nu \nabla \phi_i(x, y) \cdot \nabla \phi_j(x, y) d\Omega & (\mathbf{B}_x)_{ij} &= - \int_{\Omega} \psi_j(x, y) \frac{\partial \phi_i(x, y)}{\partial x} d\Omega & (\mathbf{B}_y)_{ij} &= - \int_{\Omega} \psi_j(x, y) \frac{\partial \phi_i(x, y)}{\partial y} d\Omega \\ \mathbf{B} &= [\mathbf{B}_x \quad \mathbf{B}_y] & \mathbf{M}_v &= \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{A} \end{bmatrix} & (\mathbf{M}_p)_{ij} &= \int_{\Omega} \psi_j(x, y) \psi_i(x, y) d\Omega \end{aligned}$$

Where:

- $\mathbf{u} \in \mathbb{R}^{2N_v}$ is the velocity vector on a fine grid;
- $\mathbf{p} \in \mathbb{R}^{N_p}$ is the pressure vector on a coarse grid;
- $\mathbf{B}_x \in \mathbb{R}^{N_p \times N_v}$ is the discrete horizontal divergence operator;
- $\mathbf{B}_y \in \mathbb{R}^{N_p \times N_v}$ is the discrete vertical divergence operator;
- $\mathbf{B} \in \mathbb{R}^{N_p \times 2N_v}$ is a discrete divergence operator, responsible for coupling pressure and velocity;
- $\mathbf{A} \in \mathbb{R}^{N_v \times N_v}$ is a discrete Laplacian operator that handles the viscosity of the fluid;
- $\mathbf{M}_v \in \mathbb{R}^{2N_v \times 2N_v}$ is the velocity energy weight matrix, used to calculate the physical size and kinetic energy of the velocity field.
- $\mathbf{M}_p \in \mathbb{R}^{N_p \times N_p}$ is the pressure mass matrix that computes the spatial overlap of pressure basis functions.

4.2 Discretization Proofs for Operators and Field Norms

Using the coordinate definitions above, we can systematically transform the three core components of the continuous inf-sup condition into algebraic matrix operations:

$$\begin{aligned}
1. \quad \int_{\Omega} q(\nabla \cdot v) d\Omega &= \left| \begin{array}{lll} q(x, y) \approx \sum_{j=1}^{N_p} p_j \psi_j(x, y) & v_x(x, y) \approx \sum_{i=1}^{N_v} u_i^x \phi_i(x, y) & \frac{\partial v_x}{\partial x} \approx \sum_{i=1}^{N_v} u_i^x \frac{\partial}{\partial x} \phi_i(x, y) \\ v(x, y) = \begin{bmatrix} v_x(x, y) \\ v_y(x, y) \end{bmatrix} & v_y(x, y) \approx \sum_{i=1}^{N_v} u_i^y \phi_i(x, y) & \frac{\partial v_y}{\partial y} \approx \sum_{i=1}^{N_v} u_i^y \frac{\partial}{\partial y} \phi_i(x, y) \end{array} \right| = \\
&= \int_{\Omega} q(x, y) \left(\begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \end{bmatrix}^T \begin{bmatrix} v_x(x, y) \\ v_y(x, y) \end{bmatrix} \right) d\Omega = \int_{\Omega} q(x, y) \left(\frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} \right) d\Omega = \\
&= \int_{\Omega} q(x, y) \frac{\partial v_x}{\partial x} d\Omega + \int_{\Omega} q(x, y) \frac{\partial v_y}{\partial y} d\Omega = \\
&= \int_{\Omega} \sum_{j=1}^{N_p} p_j \psi_j(x, y) \sum_{i=1}^{N_v} u_i^x \frac{\partial}{\partial x} \phi_i(x, y) d\Omega + \int_{\Omega} \sum_{j=1}^{N_p} p_j \psi_j(x, y) \sum_{i=1}^{N_v} u_i^y \frac{\partial}{\partial y} \phi_i(x, y) d\Omega = \\
&= \left[\sum_{j=1}^{N_p} \sum_{i=1}^{N_v} p_j \cdot \int_{\Omega} \psi_j(x, y) \frac{\partial}{\partial x} \phi_i(x, y) d\Omega \cdot u_i^x \right] + \left[\sum_{j=1}^{N_p} \sum_{i=1}^{N_v} p_j \cdot \int_{\Omega} \psi_j(x, y) \frac{\partial}{\partial y} \phi_i(x, y) d\Omega \cdot u_i^y \right] \\
&= \sum_{j=1}^{N_p} \sum_{i=1}^{N_v} p_j (B_x)_{ji} u_i^x + \sum_{j=1}^{N_p} \sum_{i=1}^{N_v} p_j (B_y)_{ji} u_i^y = \mathbf{p}^T \mathbf{B}_x \mathbf{u}_x + \mathbf{p}^T \mathbf{B}_y \mathbf{u}_y = \mathbf{p}^T [\mathbf{B}_x \quad \mathbf{B}_y] \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \end{bmatrix} = \\
&= \mathbf{p}^T \mathbf{B} \mathbf{u}
\end{aligned}$$

$$\begin{aligned}
2. \quad \|v\|_V &= \left(\int_{\Omega} (\|\nabla v_x\|^2 + \|\nabla v_y\|^2) d\Omega \right)^{1/2} = \left(\int_{\Omega} (\nabla v_x \cdot \nabla v_x + \nabla v_y \cdot \nabla v_y) d\Omega \right)^{1/2} = \\
&= \left| \begin{array}{ll} v_x(x, y) \approx \sum_{i=1}^{N_v} u_i^x \phi_i(x, y) & \nabla v_x(x, y) \approx \sum_{i=1}^{N_v} u_i^x \nabla \phi_i(x, y) \\ v_y(x, y) \approx \sum_{j=1}^{N_v} u_j^y \phi_j(x, y) & \nabla v_y(x, y) \approx \sum_{j=1}^{N_v} u_j^y \nabla \phi_j(x, y) \end{array} \right| = \\
&= \left(\int_{\Omega} \left(\sum_{i=1}^{N_v} u_i^x \nabla \phi_i(x, y) \cdot \sum_{j=1}^{N_v} u_j^x \nabla \phi_j(x, y) + \sum_{i=1}^{N_v} u_i^y \nabla \phi_i(x, y) \cdot \sum_{j=1}^{N_v} u_j^y \nabla \phi_j(x, y) \right) d\Omega \right)^{1/2} = \\
&= \left(\left[\sum_{i=1}^{N_v} \sum_{j=1}^{N_v} u_i^x \cdot \int_{\Omega} \nabla \phi_i(x, y) \cdot \nabla \phi_j(x, y) d\Omega \cdot u_j^x \right] + \sum_{i=1}^{N_v} \sum_{j=1}^{N_v} \left[u_i^y \cdot \int_{\Omega} \nabla \phi_i(x, y) \cdot \nabla \phi_j(x, y) d\Omega \cdot u_j^y \right] \right)^{1/2} = \\
&= \left(\left[\sum_{i=1}^{N_v} \sum_{j=1}^{N_v} u_i^x A_{ij} u_j^x \right] + \left[\sum_{i=1}^{N_v} \sum_{j=1}^{N_v} u_i^y A_{ij} u_j^y \right] \right)^{1/2} = \\
&= (\mathbf{u}_x^T \mathbf{A} \mathbf{u}_x + \mathbf{u}_y^T \mathbf{A} \mathbf{u}_y)^{1/2} = \left(\begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \end{bmatrix}^T \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{A} \end{bmatrix} \begin{bmatrix} \mathbf{u}_x \\ \mathbf{u}_y \end{bmatrix} \right)^{1/2} = (\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2}
\end{aligned}$$

$$\begin{aligned}
3. \quad \|q\|_Q &= \left(\int_{\Omega} q(x, y)^2 d\Omega \right)^{1/2} = \left| q(x, y) \approx \sum_{i=1}^{N_p} p_i \psi_i(x, y) \right| = \\
&= \left(\int_{\Omega} \sum_{i=1}^{N_p} p_i \psi_i(x, y) \cdot \sum_{j=1}^{N_p} p_j \psi_j(x, y) d\Omega \right)^{1/2} = \left(\int_{\Omega} \sum_{i=1}^{N_p} \sum_{j=1}^{N_p} p_i p_j \psi_j(x, y) \psi_i(x, y) d\Omega \right)^{1/2} = \\
&= \left(\sum_{i=1}^{N_p} \sum_{j=1}^{N_p} p_i \cdot \left(\int_{\Omega} \psi_j(x, y) \psi_i(x, y) d\Omega \right) \cdot p_j \right)^{1/2} = \left(\sum_{i=1}^{N_p} \sum_{j=1}^{N_p} p_i \cdot (M_p)_{ij} \cdot p_j \right)^{1/2} = (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}
\end{aligned}$$

4.3 The Final Discrete Matrix Form

By substituting our three derived algebraic expressions directly back into the continuous supremum framework, the continuous inf-sup condition simplifies into its final, discrete matrix form:

$$\exists \beta > 0 : \quad \inf_{\mathbf{p} \neq \mathbf{0}} \sup_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2} \cdot (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right) \geq \beta$$

This discrete fraction acts as a normalized compatibility test, using $\mathbf{M}_v$ and $\mathbf{M}_p$ to ensure that the fields are measured by their true physical sizes rather than arbitrary numbers. Because calculating a nested maximum and minimum over thousands of mesh nodes is impossible for a computer, we must use a mathematical shortcut to transform this fraction into a simple matrix eigenvalue problem.

4.4 Solving the Optimization Problem

The optimization problem is of the following appearance:

$$\min_{\mathbf{p} \neq \mathbf{0}} \max_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2} \cdot (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right)$$

We start with the maximization of with respect to $\mathbf{u}$:

$$\begin{aligned} \max_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2} \cdot (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right) &= \frac{1}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \max_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2}} \right) = \left| \begin{array}{l} \mathbf{M}_v = \mathbf{M}_v^{1/2} \mathbf{M}_v^{1/2} \\ \mathbf{w} = \mathbf{M}_v^{1/2} \mathbf{u}, \quad \mathbf{u} = \mathbf{M}_v^{-1/2} \mathbf{w} \end{array} \right| = \\ &= \frac{1}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \max_{\mathbf{w} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} (\mathbf{M}_v^{-1/2} \mathbf{w})}{\left((\mathbf{M}_v^{-1/2} \mathbf{w})^T \mathbf{M}_v (\mathbf{M}_v^{-1/2} \mathbf{w}) \right)^{1/2}} \right) = \\ &= \frac{1}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \max_{\mathbf{w} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{M}_v^{-1/2} \mathbf{w}}{\left(\mathbf{w}^T \mathbf{M}_v^{-1/2} \mathbf{M}_v^{1/2} \mathbf{M}_v^{1/2} \mathbf{M}_v^{-1/2} \mathbf{w} \right)^{1/2}} \right) = \\ &= \frac{1}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \max_{\mathbf{w} \neq \mathbf{0}} \frac{\left(\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p} \right)^T \mathbf{w}}{\|\mathbf{w}\|_2} = \left| \begin{array}{l} \mathbf{a} = \mathbf{c} \mathbf{x} \\ \max_{\mathbf{a} \neq \mathbf{0}} \frac{\mathbf{x}^T \mathbf{a}}{\|\mathbf{a}\|_2} = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\mathbf{c} \mathbf{x}^T \mathbf{x}}{c \|\mathbf{x}\|_2} = \|\mathbf{x}\|_2 \end{array} \right| = \\ &= \frac{1}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \|\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p}\|_2 \end{aligned}$$

With the maximization out of the way it is now possible to minimize over pressure $\mathbf{p}$:

$$\begin{aligned} \min_{\mathbf{p} \neq \mathbf{0}} \frac{\|\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p}\|_2}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} &\longrightarrow \left(\min_{\mathbf{p} \neq \mathbf{0}} \frac{\|\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p}\|_2}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right)^2 = \min_{\mathbf{p} \neq \mathbf{0}} \frac{\left(\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p} \right)^T \left(\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p} \right)}{\mathbf{p}^T \mathbf{M}_p \mathbf{p}} = \\ &= \min_{\mathbf{p} \neq \mathbf{0}} \frac{\mathbf{p}^T \mathbf{B} \mathbf{M}_v^{-1/2} \mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p}}{\mathbf{p}^T \mathbf{M}_p \mathbf{p}} = \min_{\mathbf{p} \neq \mathbf{0}} \frac{\mathbf{p}^T (\mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T) \mathbf{p}}{\mathbf{p}^T \mathbf{M}_p \mathbf{p}} = \left| \begin{array}{l} R(\mathbf{A}, \mathbf{B}; \mathbf{x}) = \frac{\mathbf{x}^T \mathbf{A} \mathbf{x}}{\mathbf{x}^T \mathbf{B} \mathbf{x}} \\ \mathbf{q} = \mathbf{M}_p^{1/2} \mathbf{p} \\ \mathbf{p} = \mathbf{M}_p^{-1/2} \mathbf{q} \end{array} \right| = \\ &= \min_{\mathbf{q} \neq \mathbf{0}} \frac{\left(\mathbf{M}_p^{-1/2} \mathbf{q} \right)^T (\mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T) \left(\mathbf{M}_p^{-1/2} \mathbf{q} \right)}{\left(\mathbf{M}_p^{-1/2} \mathbf{q} \right)^T \mathbf{M}_p \left(\mathbf{M}_p^{-1/2} \mathbf{q} \right)} = \min_{\mathbf{q} \neq \mathbf{0}} \frac{\mathbf{q}^T \left(\mathbf{M}_p^{-1/2} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T \mathbf{M}_p^{-1/2} \right) \mathbf{q}}{\mathbf{q}^T \mathbf{q}} = \end{aligned}$$

$$= \left| \begin{array}{c} \text{Rayleigh quotient:} \\ R(\mathbf{K}; \mathbf{x}) = \frac{\mathbf{x}^T \mathbf{K} \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \\ \mathbf{K} = \mathbf{M}_p^{-1/2} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T \mathbf{M}_p^{-1/2} \end{array} \right| = \min_{\mathbf{q} \neq \mathbf{0}} R(\mathbf{K}; \mathbf{q}) = \lambda_{\min}(\mathbf{K})$$

To compute minimum of the Rayleigh quotient we would need to compute its minimal eigenvalue and get rid of the substitution $\mathbf{q}$:

$$\mathbf{K} \mathbf{q} = \lambda \mathbf{q}$$

$$\begin{aligned} \mathbf{K} \mathbf{M}_p^{1/2} \mathbf{p} &= \lambda \mathbf{M}_p^{1/2} \mathbf{p} \quad | \cdot \mathbf{M}_p^{-1/2} \\ \mathbf{M}_p^{-1/2} \mathbf{K} \mathbf{M}_p^{1/2} \mathbf{p} &= \lambda \mathbf{M}_p^{-1/2} \mathbf{M}_p^{1/2} \mathbf{p} \\ \mathbf{M}_p^{-1/2} \left(\mathbf{M}_p^{-1/2} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T \mathbf{M}_p^{-1/2} \right) \mathbf{M}_p^{1/2} \mathbf{p} &= \lambda \mathbf{p} \\ (\mathbf{M}_p^{-1} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T) \mathbf{p} &= \lambda \mathbf{p} \\ \lambda_{\min}(\hat{\mathbf{K}}), \quad \text{where: } \hat{\mathbf{K}} &= \mathbf{M}_p^{-1} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T \end{aligned}$$

We also have to revert the squaring of the term used earlier:

$$\begin{aligned} \left(\min_{\mathbf{p} \neq \mathbf{0}} \frac{\|\mathbf{M}_v^{-1/2} \mathbf{B}^T \mathbf{p}\|_2}{(\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right)^2 &= \lambda_{\min}(\hat{\mathbf{K}}) \\ \min_{\mathbf{p} \neq \mathbf{0}} \max_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2} \cdot (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right) &= \sqrt{\lambda_{\min}(\hat{\mathbf{K}})} \end{aligned}$$

4.5 Numerical Stability Assessment

Lastly, we can look at the stability condition in its final, practical form. In a two-dimensional simulation, the velocity field is split into horizontal and vertical components. Because of this, the global divergence matrix $\mathbf{B}$ and the velocity matrix $\mathbf{M}_v$ must be expanded into their individual block components $\mathbf{B}_x$, $\mathbf{B}_y$, and the velocity stiffness matrix $\mathbf{A}$.

Substituting these block matrices into our eigenvalue problem yields the fully expanded discrete inf-sup condition:

$$\begin{aligned} \exists \beta > 0 : \quad \inf_{\mathbf{p} \neq \mathbf{0}} \sup_{\mathbf{u} \neq \mathbf{0}} \left(\frac{\mathbf{p}^T \mathbf{B} \mathbf{u}}{(\mathbf{u}^T \mathbf{M}_v \mathbf{u})^{1/2} \cdot (\mathbf{p}^T \mathbf{M}_p \mathbf{p})^{1/2}} \right) &= \sqrt{\lambda_{\min}(\hat{\mathbf{K}})} \geq \beta \\ \exists \beta > 0 : \quad \sqrt{\lambda_{\min}(\mathbf{M}_p^{-1} \mathbf{B} \mathbf{M}_v^{-1} \mathbf{B}^T)} &= \sqrt{\lambda_{\min} \left(\mathbf{M}_p^{-1} \begin{bmatrix} \mathbf{B}_x & \mathbf{B}_y \end{bmatrix} \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{A} \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{B}_x^T \\ \mathbf{B}_y^T \end{bmatrix} \right)} \geq \beta \\ \exists \beta > 0 : \quad \sqrt{\lambda_{\min}(\mathbf{M}_p^{-1} (\mathbf{B}_x \mathbf{A}^{-1} \mathbf{B}_x^T + \mathbf{B}_y \mathbf{A}^{-1} \mathbf{B}_y^T))} &\geq \beta \end{aligned}$$

The minimum eigenvalue $\lambda_{\min}$ serves as a direct numerical test for the stability of our chosen velocity and pressure finite element grids. To verify if a grid combination satisfies the discrete inf-sup condition, we look at how $\lambda_{\min}$ behaves as the mesh is refined:

- **Stable Elements:** If the chosen grid pair is stable, $\lambda_{\min}$ will level off and remain strictly greater than a positive constant ($\lambda_{\min} \geq \beta^2 > 0$) even on very fine meshes, ensuring stability of the mixed finite element discretization.

- **Unstable Elements:** If the grid pair is unstable, $\lambda_{\min}$ will continuously drop toward zero as the mesh size decreases. This acts as a clear warning that the simulation will suffer from spurious pressure oscillations.

Mass Matrix $\mathbf{M}_p$

To check our results numerically we will have to construct the pressure mass matrix as well:

$$\begin{aligned} (\mathbf{M}_p)_{ij} &= \int_{\Omega} \psi_j(x, y) \psi_i(x, y) d\Omega = \sum_T \int_T \psi_j(x, y) \psi_i(x, y) dx dy = \sum_T \int_{\hat{T}} \hat{\psi}_j(\xi_1, \xi_2) \hat{\psi}_i(\xi_1, \xi_2) |\mathbf{J}_T| d\xi_1 d\xi_2 \\ &= \sum_T |\mathbf{J}_T| \int_{\hat{T}} \hat{\psi}_j \hat{\psi}_i d\xi_1 d\xi_2 \end{aligned}$$

We can now turn to the local mass matrix:

$$\begin{aligned} \mathbf{M}_p^{loc} &= |\mathbf{J}_T| \int_{\hat{T}} \hat{\psi}_j \hat{\psi}_i d\xi_1 d\xi_2 = |\mathbf{J}_T| \int_{\hat{T}} \begin{bmatrix} \hat{\psi}_1 \hat{\psi}_1 & \hat{\psi}_1 \hat{\psi}_2 & \hat{\psi}_1 \hat{\psi}_3 \\ \hat{\psi}_2 \hat{\psi}_1 & \hat{\psi}_2 \hat{\psi}_2 & \hat{\psi}_2 \hat{\psi}_3 \\ \hat{\psi}_3 \hat{\psi}_1 & \hat{\psi}_3 \hat{\psi}_2 & \hat{\psi}_3 \hat{\psi}_3 \end{bmatrix} d\xi_1 d\xi_2 = \begin{vmatrix} \hat{\psi}_1 = 1 - \xi_1 - \xi_2 \\ \hat{\psi}_2 = \xi_1 \\ \hat{\psi}_3 = \xi_2 \end{vmatrix} = \\ &= |\mathbf{J}_T| \int_0^1 \int_0^{1-\xi_1} \begin{bmatrix} (1 - \xi_1 - \xi_2)(1 - \xi_1 - \xi_2) & \xi_1(1 - \xi_1 - \xi_2) & \xi_2(1 - \xi_1 - \xi_2) \\ \xi_1(1 - \xi_1 - \xi_2) & \xi_1 \xi_1 & \xi_2 \xi_1 \\ \xi_2(1 - \xi_1 - \xi_2) & \xi_2 \xi_1 & \xi_2 \xi_2 \end{bmatrix} d\xi_2 d\xi_1 = \\ &= \frac{|\mathbf{J}_T|}{24} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \end{aligned}$$

Numerical Verification of the Discrete Inf-Sup Condition

We have already established that the P_1 -iso- P_1 pairing is stable if and only if the minimum eigenvalue $\lambda_{\min}$ of

$$\hat{\mathbf{K}} = \mathbf{M}_p^{-1} (\mathbf{B}_x \mathbf{A}^{-1} \mathbf{B}_x^T + \mathbf{B}_y \mathbf{A}^{-1} \mathbf{B}_y^T)$$

remains bounded away from zero as the mesh is refined. We verify this directly by solving the equivalent generalized eigenvalue problem $\mathbf{K}\mathbf{p} = \lambda \mathbf{M}_p \mathbf{p}$, with $\mathbf{K} = \mathbf{B}_x \mathbf{A}^{-1} \mathbf{B}_x^T + \mathbf{B}_y \mathbf{A}^{-1} \mathbf{B}_y^T$, over six uniform refinement levels of a test domain. Since $\mathbf{A}$ is never needed in inverted form — only $\mathbf{A}^{-1} \mathbf{B}_x^T$ and $\mathbf{A}^{-1} \mathbf{B}_y^T$, i.e. N_p right-hand sides with $N_p \ll N_v$ — $\mathbf{A}$ is factorized once via sparse LU and solved against these N_p columns rather than inverted densely.

Table 1: Discrete inf-sup eigenvalue $\lambda_{\min}$ and stability constant $\beta = \sqrt{\lambda_{\min}}$ across mesh refinement.

Level	Mesh Size	N_v	N_p	$\lambda_{\min}$	β	$\Delta \lambda_{\min}$
0	$h_0/1$	57	20	0.23319	0.4829	—
1	$h_0/2$	185	57	0.13443	0.3666	0.0988
2	$h_0/4$	657	185	0.09448	0.3074	0.0399
3	$h_0/8$	2465	657	0.08640	0.2939	0.0081
4	$h_0/16$	9537	2465	0.08223	0.2868	0.0042
5	$h_0/32$	37505	9537	0.08095	0.2845	0.0013

Table 1 and Figure 2 show that as we make the grid significantly finer, the minimum eigenvalue $\lambda_{\min}$ drops from 0.2332 to 0.0809. The successive differences ($\Delta \lambda_{\min}$) decrease rapidly, indicating that the eigenvalue approaches a positive limiting value rather than converging to zero. This behavior provides clear, practical proof that our P_1 -iso- P_1 setup is numerically stable and successfully satisfies the LBB condition.

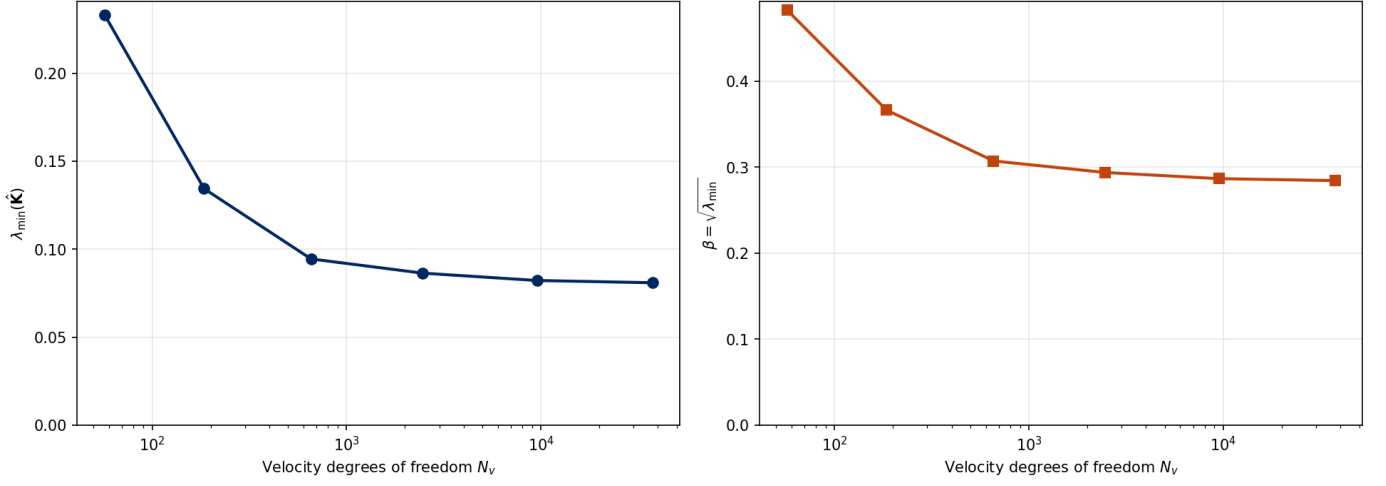
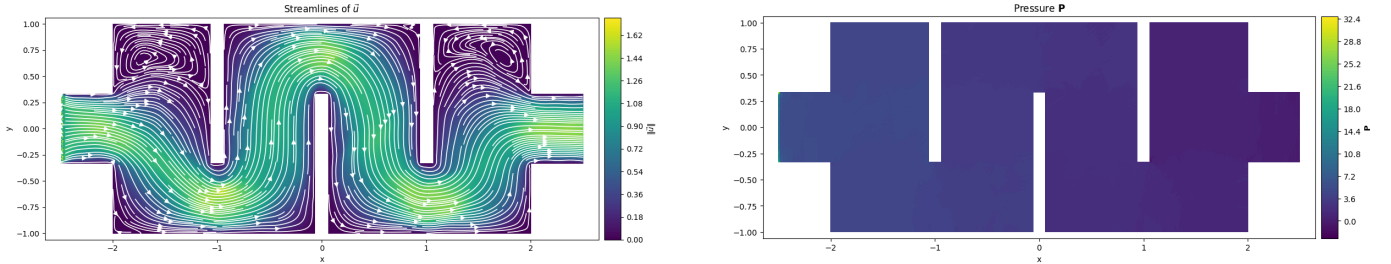


Figure 2: Discrete inf-sup eigenvalue $\lambda_{\min}$ (left) and stability constant β (right) as a function of velocity degrees of freedom N_v , over six uniform refinement levels.

5 Numerical Experiments and Applications

In this section, we validate our P_1 -iso- P_1 finite element solver using two distinct fluid engineering case studies: an *Exchanger Device* and a *Winding Pipe*. Both configurations operate under a creeping flow regime ($\text{Re} \ll 1$), where viscous forces dominate.

Exchanger Device



(a) Computed Velocity Streamline Vector Paths.

(b) Continuous Piecewise Linear Pressure Contour Map.

Figure 3: Complete Stokes flow simulation inside the *Exchanger Device* domain.

The numerical results inside the *Exchanger Device* reveal distinct kinematic adjustments driven by the variable cross-sectional area of the channel network. As observed in the velocity streamline distributions (Figure 3a), the fluid velocity increases significantly at the turns, where the geometry narrows relative to the wider vertical passages. To preserve mass conservation under the incompressibility constraint $\nabla \cdot \mathbf{u} = 0$, the velocity field must accelerate through these local contractions, creating higher localized velocity gradients along the curved inner walls.

Furthermore, the simulation clearly shows fluid swirls (recirculation zones) inside the dead-end, cut-off extensions. Because Stokes flow has no inertia, the fluid balance simplifies to a strict equilibrium between pressure gradients and viscous friction:

$$-\nu \Delta \mathbf{u} + \nabla p = \mathbf{0}$$

As a result, the main stream takes the path of least resistance through the primary central channels. However, as this fast-moving core flow sweeps past the stagnant fluid trapped in the side cavities, it shears against it. This boundary shear transfers momentum across the open gap, dragging the passive fluid along and creating slow, circular vortices inside the extensions. The computed velocity field exhibits well-resolved stagnation regions and sharp shear interfaces, indicating stable and physically consistent behavior of the mixed finite element discretization.

Winding Pipe

The *Winding Pipe* models an internal distribution manifold containing geometric obstacles designed to alter flow paths. This setup presents a challenging test case for mass conservation preservation across sudden transitions.

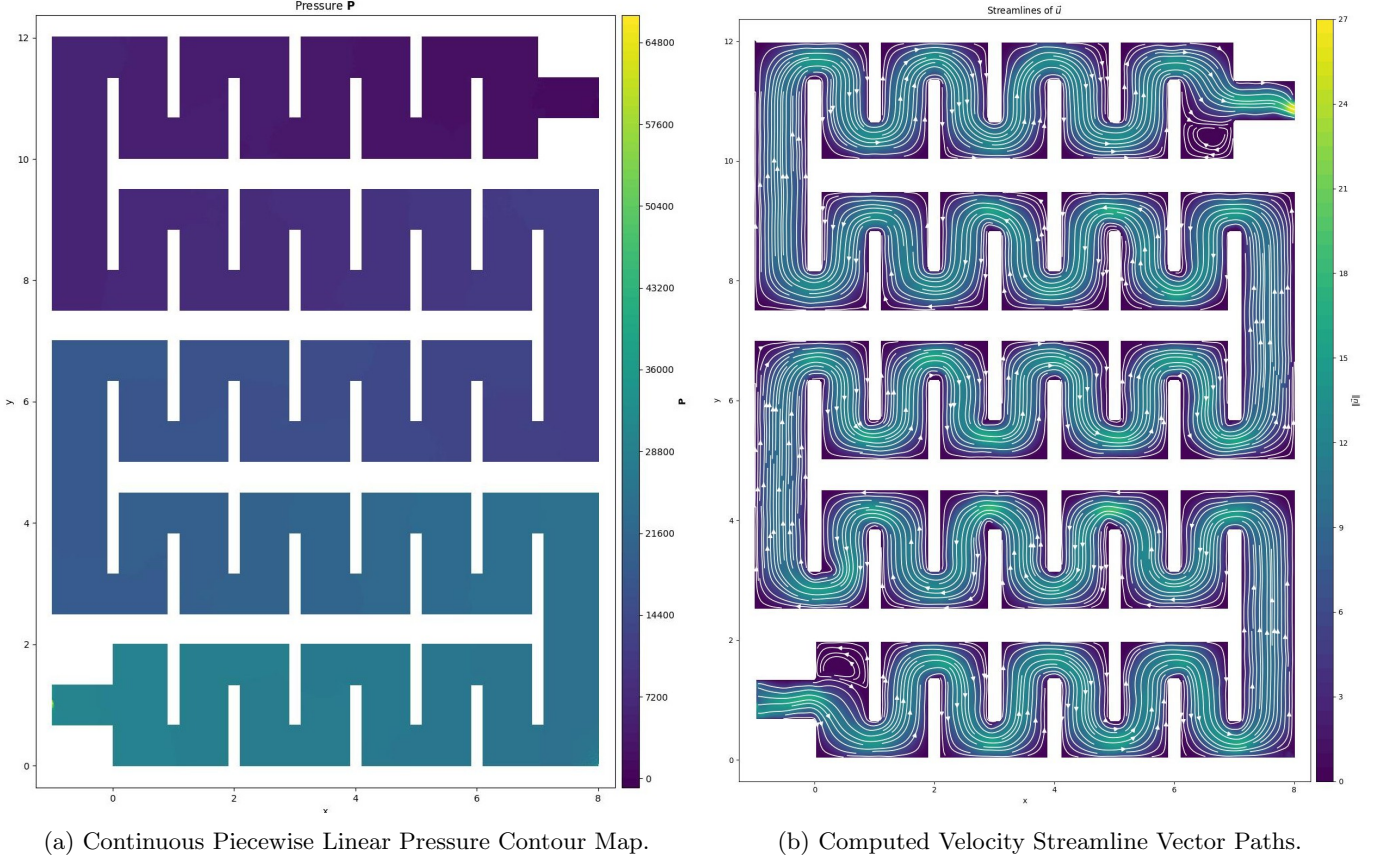
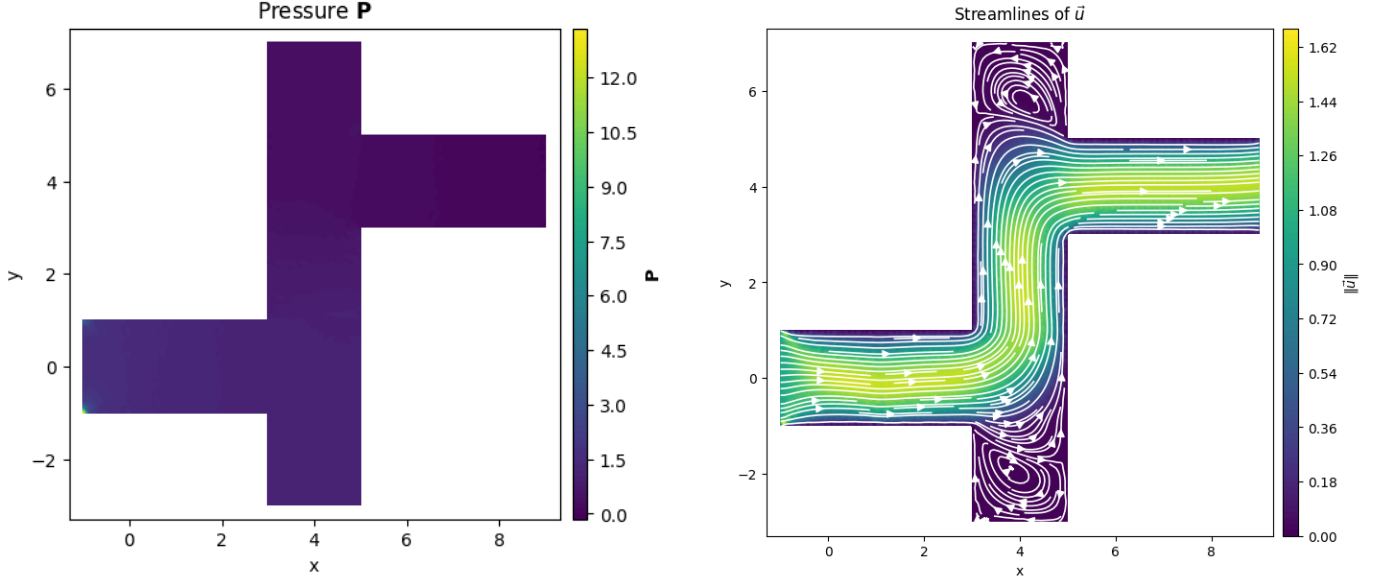


Figure 4: Complete Stokes flow simulation inside the *Winding Pipe* domain.

As observed in Figure 5, the velocity streamlines adapt smoothly around the internal channel obstacles without displaying spurious velocity vector spikes. Concurrently, the pressure distribution field (Figure 4a) displays a continuous linear gradient drop, confirming the inf-sup spatial stability of the dual-grid discretization matrix configuration.

6 Convergence of Numerical Solutions

Convergence Analysis via a Reference Solution on a Z-Shaped Channel Device



(a) Continuous Piecewise Linear Pressure Contour Map.

(b) Computed Velocity Streamline Vector Paths.

Figure 5: Complete Stokes flow simulation inside the *Z-Shaped Channel Device* domain.

To investigate the convergence behavior of the P1-iso-P1 discretization, a reference solution was first computed on the finest available mesh, obtained after seven uniform refinements of the initial domain geometry. Solutions on coarser meshes were then compared against this reference solution.

Since the discrete velocity and pressure spaces have different dimensions on different refinement levels, the solutions cannot be compared directly. Therefore, each coarse-grid solution was recursively transferred to the reference mesh using the finite-element embedding operator. For a solution computed on refinement level i , the embedding matrices corresponding to all subsequent refinements were applied successively,

$$\mathbf{u}_{\text{emb}}^{(i)} = \mathbf{E}_N \cdots \mathbf{E}_{i+1} \mathbf{E}_i \mathbf{u}^{(i)},$$

and analogously for the pressure field. After each embedding step, the mesh was refined and the next embedding matrix was constructed on the newly generated mesh. This recursive procedure produced velocity and pressure approximations living in the same finite-element spaces as the reference solution.

The errors were then defined by

$$\mathbf{e}_{u_x} = \mathbf{u}_{x,\text{emb}} - \mathbf{u}_{x,\text{ref}}, \quad \mathbf{e}_{u_y} = \mathbf{u}_{y,\text{emb}} - \mathbf{u}_{y,\text{ref}},$$

and

$$\mathbf{e}_p = \mathbf{p}_{\text{emb}} - \mathbf{p}_{\text{ref}}.$$

Their discrete L^2 -norms were evaluated using the mass matrices corresponding to the fine and coarse spaces on the reference mesh,

$$\|\mathbf{e}_u\|_{L^2(\Omega)} = \sqrt{\mathbf{e}_{u_x}^T \mathbf{M}_{\text{fine}} \mathbf{e}_{u_x} + \mathbf{e}_{u_y}^T \mathbf{M}_{\text{fine}} \mathbf{e}_{u_y}},$$

and

$$\|\mathbf{e}_p\|_{L^2(\Omega)} = \sqrt{\mathbf{e}_p^T \mathbf{M}_{\text{coarse}} \mathbf{e}_p}.$$

Finally, the experimental convergence rates were estimated from two consecutive refinement levels according to

$$r = \frac{\log(E_h/E_{h/2})}{\log(2)},$$

where E_h denotes the corresponding L^2 -error.

Table 2: Stokes FEM Error Convergence Analysis on the Z-Shaped Channel Geometric Mesh.

Level	Mesh Size	Velocity DOFs	Pressure DOFs	L^2 Error u	Rate (r_u)	L^2 Error p	Rate (r_p)
0	$h_0/1$	114	20	39.5400	—	4826.3	—
1	$h_0/2$	370	57	27.2150	0.539	4804.2	0.007
2	$h_0/4$	1314	185	11.9470	1.188	4725.1	0.024
3	$h_0/8$	4930	657	4.9725	1.265	4571.5	0.048
4	$h_0/16$	19074	2465	2.3325	1.092	4266.3	0.100
5	$h_0/32$	75010	9537	1.2049	0.953	3657.0	0.222
6	$h_0/64$	297474	37505	0.7305	0.722	2438.1	0.585

Table 2 demonstrates a consistent reduction of both the velocity and pressure errors as the mesh is refined, indicating convergence of the numerical solution toward the reference solution. The velocity error decreases from 39.5400 on the coarsest mesh to 0.7305 on the finest mesh considered, with observed convergence rates oscillating between 0.54 and 1.27. The pressure error also decreases, although at a significantly slower rate, from 4826.3 to 2438.1. This behavior is expected, as the pressure approximation is computed on the coarser mesh of the P1-iso-P1 discretization and therefore converges more slowly than the velocity field. Additionally, the re-entrant corners inherent to this specific device geometry introduce flow singularities that naturally degrade the theoretical asymptotic convergence rates. Overall, the results confirm that mesh refinement improves the accuracy of both solution components and provide numerical evidence for the convergence of the implemented Stokes finite element method.

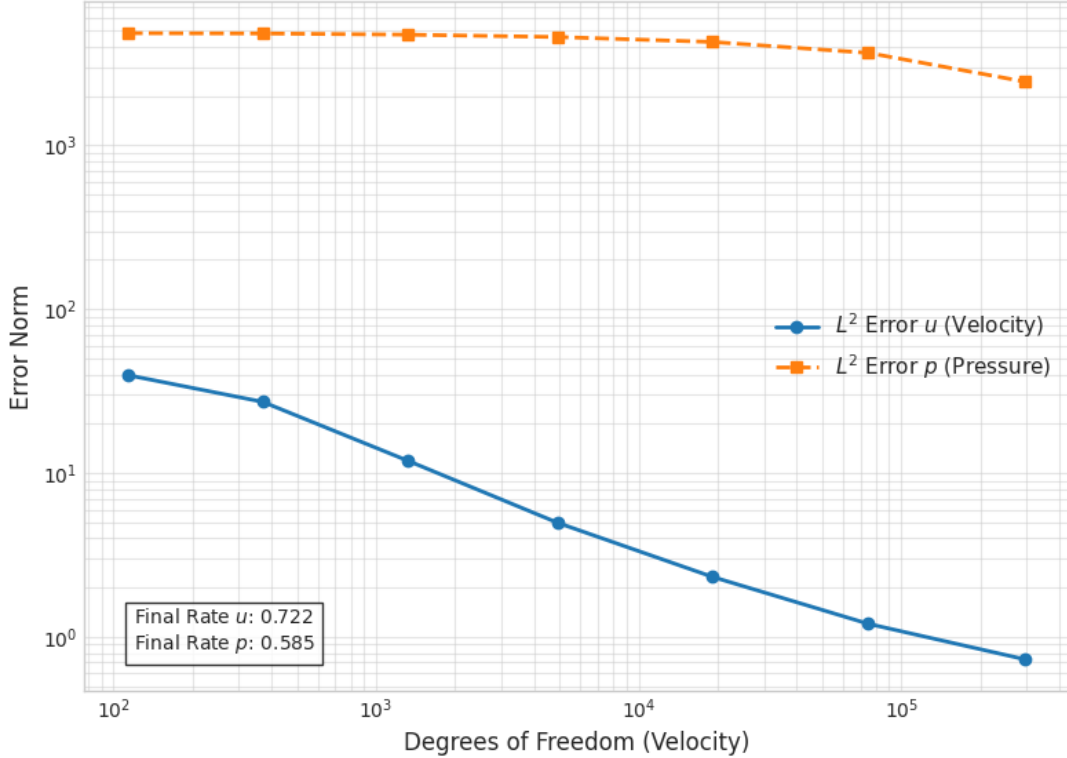
Figure 6: Log-log plot of the L^2 error norms for velocity and pressure against the velocity degrees of freedom.

Figure 6 visualizes the convergence behavior detailed in Table 2 by plotting the L^2 error norms against the velocity degrees of freedom on a logarithmic scale. The solid blue line demonstrates a steady, near-linear decrease in the velocity error, ultimately achieving a final convergence rate of $r_u = 0.722$. In contrast, the dashed orange line representing the pressure error starts at a significantly higher magnitude and exhibits a much flatter trajectory, finalizing at a rate of $r_p = 0.585$. This visual disparity emphasizes the slower convergence of the pressure field inherent to the P1-iso-P1 element pair, particularly when subjected to the geometric singularities of the Z-shaped domain.

7 Introduction to Reduced-Order Modeling

The finite element discretization developed in the preceding sections yields a numerically reliable, but computationally expensive, *full-order model* (FOM): even at moderate refinement levels the P_1 -iso- P_1 discretization produces velocity systems with $\mathcal{O}(10^5)$ degrees of freedom (Table 2), and every change in the physical configuration requires reassembling and resolving the full saddle-point system. This cost becomes prohibitive in *many-query* settings — parameter studies, optimization loops, or real-time control — where the Stokes system must be solved repeatedly for many instances of a parameter vector $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p$. In the present study, $\boldsymbol{\mu}$ parametrizes a spatially varying kinematic viscosity field, and solving the FOM anew for every parameter instance is computationally wasteful: nearby parameters produce similar flow fields, so the resulting solution manifold,

$$\mathcal{M} = \{(\mathbf{u}_h(\boldsymbol{\mu}), p_h(\boldsymbol{\mu})) : \boldsymbol{\mu} \in \mathcal{P}\},$$

possesses low intrinsic dimension despite living in a high-dimensional discrete space, and can be well approximated by a low-dimensional linear subspace. This observation is the central premise of *reduced-order modeling* (ROM): rather than repeatedly solving the full-order system, one projects the problem onto a much smaller subspace and solves a reduced Galerkin system instead.

The construction of this reduced system naturally splits into two stages. In an *offline* stage, performed once, a set of representative FOM solutions is precomputed and compressed into a low-dimensional reduced basis; any parameter-independent quantities needed to assemble the reduced operators are also built at this stage, regardless of computational cost, since it is paid only once. In the subsequent *online* stage, a new parameter instance $\boldsymbol{\mu}$ is handled by assembling and solving only the small reduced system, at a cost independent of the underlying mesh resolution N . This offline–online decomposition is what ultimately makes ROM attractive for many-query settings: the expensive part of the computation is amortized once, while each additional query becomes essentially free by comparison.

8 Parametrized Problem Formulation

The *exchanger device* consists of a pipe-like geometry in which the temperature, and consequently the viscosity, varies along the flow direction. Rather than resolving this dependence by solving the coupled advection–diffusion equation for the temperature field, the viscosity distribution is approximated directly from values prescribed at *five* representative locations inside the domain and reconstructed throughout Ω by *inverse-distance (Shepard) interpolation*. This parameterization is what makes the problem amenable to reduced-order modeling.

8.1 The Viscosity Parameter Vector $\boldsymbol{\mu}$

To enable an efficient reduced-order formulation, the spatially varying kinematic viscosity is parameterized by a finite set of basis functions. Five normalized Shepard basis functions $\{\psi_k(\mathbf{x})\}_{k=1}^5$ are constructed from prescribed locations distributed over the computational domain. The viscosity field is then represented as

$$\nu(\mathbf{x}; \boldsymbol{\mu}) = \sum_{k=1}^p \mu_k \psi_k(\mathbf{x}), \quad \psi_k(\mathbf{x}) = \frac{w_k(\mathbf{x})}{\sum_{j=1}^p w_j(\mathbf{x})}, \quad w_k(\mathbf{x}) = \frac{1}{\max(d_k, \epsilon)^\gamma}, \quad d_k = \|\mathbf{x} - \mathbf{x}_k\|_2,$$

where

- $\boldsymbol{\mu} = [\mu_1 \ \mu_2 \ \mu_3 \ \mu_4 \ \mu_5]^T \in \mathcal{P} \subset \mathbb{R}^5$ is the vector of viscosity values assigned at the control points,
- $p = 5$ is the number of control points distributed across the domain,
- $\mathbf{x}_k \in \Omega$ is the fixed spatial position of the k -th control point,
- $\gamma > 0$ is a power parameter controlling the decay of influence with distance.

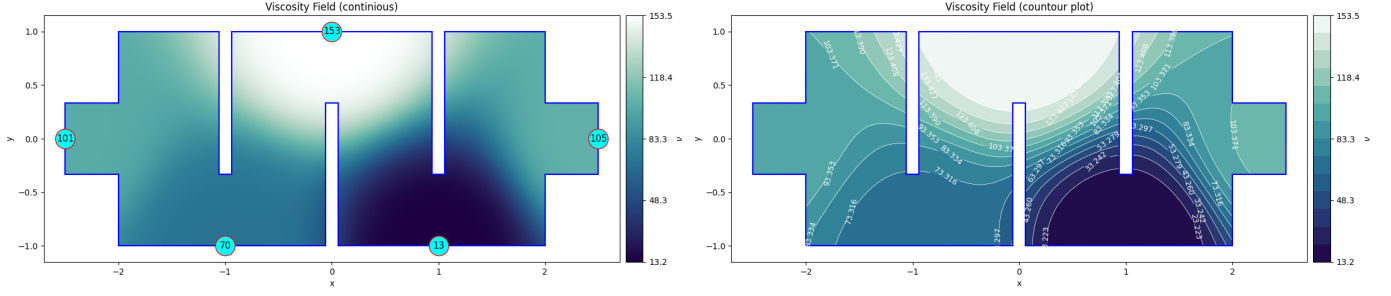


Figure 7: The viscosity field $\nu(\mathbf{x}; \boldsymbol{\mu} = [101.014 \ 70.127 \ 153.464 \ 13.205 \ 104.624]^T)$ across the *exchanger device's* domain.

Left: Continuous viscosity field with labeled evaluation points.

Right: Corresponding contour plot showing lines of constant viscosity.

8.2 Affine Decomposition of the Viscous Operator

Since the viscosity field enters the weak form linearly, the corresponding viscous operator inherits an affine parameter dependence. Under this affine structure, the bilinear form decomposes into parameter-independent spatial terms scaled by the parameter coefficients μ_k . Consequently, these spatial operators can be precomputed once and reused for every $\boldsymbol{\mu} \in \mathcal{P}$, forming the foundation of the reduced-order formulation. Since the assembly treats the viscosity as piecewise constant on each element — evaluated once at the element's centroid $\mathbf{x}_T$, exactly as in the numerical assembly routine — the affine structure of $\nu(\mathbf{x}; \boldsymbol{\mu})$ propagates directly to the global stiffness matrix:

$$\begin{aligned} \mathbf{A}(\boldsymbol{\mu}) &= \sum_{T \in \mathcal{T}_h} \nu(\mathbf{x}_T; \boldsymbol{\mu}) \mathbf{A}_T^{loc} = \sum_{T \in \mathcal{T}_h} \left(\sum_{k=1}^5 \mu_k \psi_k(\mathbf{x}_T) \right) \mathbf{A}_T^{loc} = \sum_{k=1}^5 \mu_k \sum_{T \in \mathcal{T}_h} \psi_k(\mathbf{x}_T) \mathbf{A}_T^{loc} \\ &= \sum_{k=1}^5 \mu_k \mathbf{A}_k, \end{aligned}$$

where $\mathbf{A}_T^{loc}$ denotes the element stiffness matrix scattered into the global index set (the same local matrix as in Section 3.2, but with the viscosity factor left out), and

$$\mathbf{A}_k := \sum_{T \in \mathcal{T}_h} \psi_k(\mathbf{x}_T) \mathbf{A}_T^{loc}$$

is entirely independent of $\boldsymbol{\mu}$. Each $\mathbf{A}_k$ is obtained by running the ordinary assembly routine once with ψ_k itself substituted in place of the viscosity field, so five parameter-independent matrices $\mathbf{A}_1, \dots, \mathbf{A}_5$ are assembled and stored once, offline; online, $\mathbf{A}(\boldsymbol{\mu})$ for any new parameter instance reduces to a five-term scalar-weighted sum.

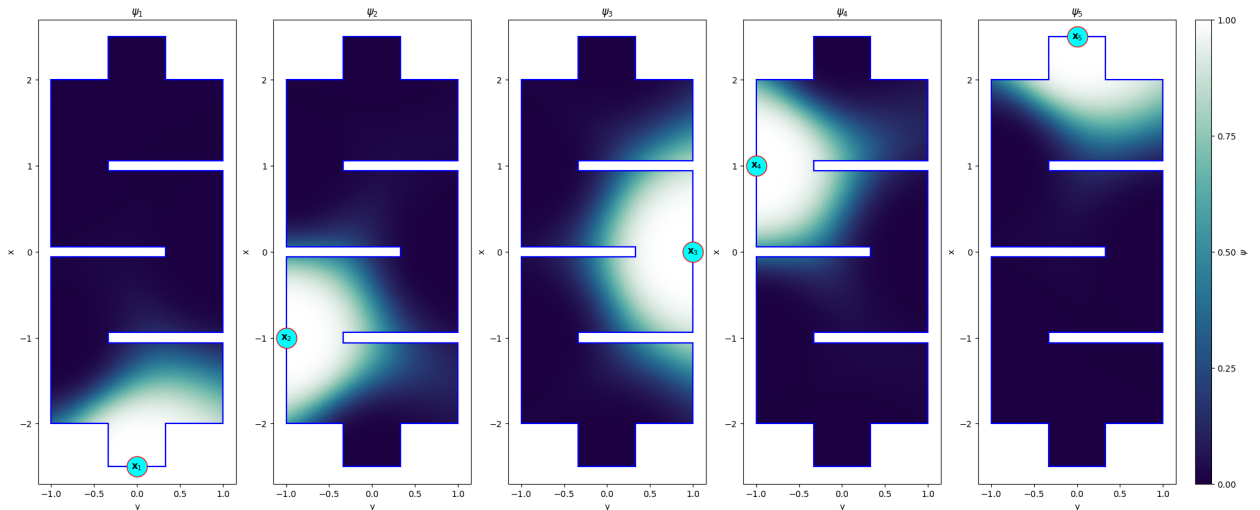


Figure 8: The viscosity basis functions $\{\psi_i\}_{i=1}^5$ across the *exchanger device's* domain.

9 Data Generation

In this section, we detail the data generation pipeline required to construct the reduced-order model and address the computational and mathematical caveats inherent to the process.

9.1 Full-Order Snapshot Collection

To generate the training dataset of $n_s = 100$ snapshots, we evaluate the high-fidelity model over a discrete parameter set $\Xi = \{\boldsymbol{\mu}^{(1)}, \dots, \boldsymbol{\mu}^{(n_s)}\} \subset \mathcal{P}$. To ensure a robust, space-filling exploration of the five-dimensional parameter domain, these instances were generated using *Latin Hypercube Sampling* (LHS) over the uniform bounds $\mathcal{P} = [10, 200]^5$.

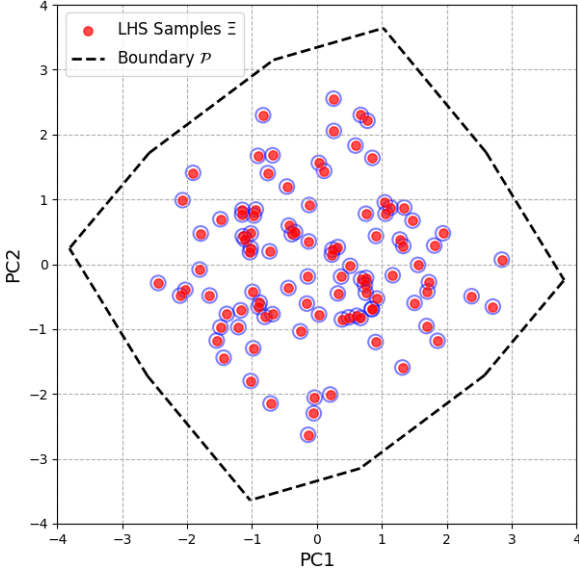


Figure 9: Down-projection of LHS parameter set Ξ bounded by the convex hull of the parameter space $\mathcal{P} = [10, 200]^5$.

To visually verify the spatial distribution of the sampling points across the 5D domain, the parameters were standardized and projected onto a two-dimensional subspace using *principal component analysis* (PCA). Figure 9 displays the down-projected snapshots Ξ alongside the convex hull of the parameter space $\mathcal{P}$. The uniform dispersion of points across the interior of the boundary confirms that the LHS scheme achieves excellent space-filling properties, effectively capturing the global parameter behavior without leaving unrepresented regions or forming dense clusters.

Next, by exploiting the affine decomposition of the viscous operator,

$$A(\boldsymbol{\mu}) = \sum_{k=1}^5 \mu_k A_k,$$

we eliminate the computationally prohibitive requirement of generating and storing n_s dense spatial viscosity fields, and instead assemble the global stiffness matrix for any configuration via a scalar combination of the five precomputed operators A_k .

The full-order saddle-point system was then solved for each $\boldsymbol{\mu}^{(i)} \in \Xi$ utilizing a *parallelized computational framework* distributed across 10 CPU cores to obtain $\{(\mathbf{u}_h(\boldsymbol{\mu}^{(1)}), p_h(\boldsymbol{\mu}^{(1)})), \dots, (\mathbf{u}_h(\boldsymbol{\mu}^{(n_s)}), p_h(\boldsymbol{\mu}^{(n_s)}))\}$. Aggregating these discrete solutions produces the global snapshot matrices $\mathbf{S}_{u_x}, \mathbf{S}_{u_y} \in \mathbb{R}^{N_v \times n_s}$ and $\mathbf{S}_p \in \mathbb{R}^{N_p \times n_s}$. To be more specific:

$$\begin{aligned} \mathbf{S}_{u_x} &= \begin{bmatrix} \mathbf{u}_x^{(1)} & \dots & \mathbf{u}_x^{(100)} \end{bmatrix} \in \mathbb{R}^{96817 \times 100} \\ \mathbf{S}_{u_y} &= \begin{bmatrix} \mathbf{u}_y^{(1)} & \dots & \mathbf{u}_y^{(100)} \end{bmatrix} \in \mathbb{R}^{96817 \times 100} \\ \mathbf{S}_p &= \begin{bmatrix} \mathbf{p}^{(1)} & \dots & \mathbf{p}^{(100)} \end{bmatrix} \in \mathbb{R}^{24473 \times 100} \end{aligned}$$

The total velocity snapshot matrix $\mathbf{S}_u$ is also introduced to make the computations easier later:

$$\mathbf{S}_u = \begin{bmatrix} \mathbf{S}_{u_x} \\ \mathbf{S}_{u_y} \end{bmatrix} \in \mathbb{R}^{2N_v \times n_s}, \quad \mathbf{S}_u = \begin{bmatrix} \mathbf{u}^{(1)} & \dots & \mathbf{u}^{(100)} \end{bmatrix} \in \mathbb{R}^{193634 \times 100}$$

9.2 Supremizer Enrichment for Inf-Sup Stability

Although the full-order P_1 -iso- P_1 discretization satisfies the discrete inf-sup condition, this property is not automatically inherited by independently constructed reduced velocity and pressure spaces. A reduced basis obtained solely by applying *proper orthogonal decomposition* (POD) separately to the velocity and pressure snapshots may violate the discrete LBB condition at the reduced level, producing an *unstable reduced-order model* even though the full-order spaces it was built from are themselves stable.

Stability is recovered by enriching the reduced velocity space with *supremizer modes*, which are derived as follows.

For a given pressure field $q \in Q$, the velocity supremizer $t \in V$ maximizes the discrete velocity-pressure divergence coupling $\int_{\Omega} q (\nabla \cdot v) d\Omega$. By the Riesz representation theorem, t uniquely satisfies the variational problem:

$$\int_{\Omega} \nabla t : \nabla v d\Omega = \int_{\Omega} q (\nabla \cdot v) d\Omega \quad \forall v \in V$$

t , test functions v , and pressure q can be expanded in terms of their finite element basis functions and later substituted back into the variational problem:

$$t = \sum_{j=1}^{2N_v} \mathbf{t}_j \phi_j, \quad v = \sum_{i=1}^{2N_v} \mathbf{v}_i \phi_i, \quad q = \sum_{k=1}^{N_p} \mathbf{p}_k \psi_k,$$

$$\sum_{i=1}^{2N_v} \sum_{j=1}^{2N_v} \mathbf{v}_i \left(\int_{\Omega} \nabla \phi_i : \nabla \phi_j d\Omega \right) \mathbf{t}_j = \sum_{i=1}^{2N_v} \sum_{k=1}^{N_p} \mathbf{v}_i \left(\int_{\Omega} \psi_k (\nabla \cdot \phi_i) d\Omega \right) \mathbf{p}_k$$

$$\mathbf{v}^T \mathbf{X}_u \mathbf{t} = \mathbf{v}^T \mathbf{B}_h^T \mathbf{p}$$

Since this relation holds for arbitrary $\mathbf{v}$, we obtain the discrete system

$$\mathbf{X}_u \mathbf{t} = \mathbf{B}_h^T \mathbf{p} \implies \mathbf{t} = \mathbf{X}_u^{-1} \mathbf{B}_h^T \mathbf{p}.$$

Now, for every pressure snapshot $\mathbf{p}_h(\boldsymbol{\mu}^{(k)})$, $k = 1, \dots, n_s$, the corresponding algebraic supremizer solution $\mathbf{t}^{(k)}$ is obtained by solving the auxiliary linear system

$$\mathbf{X}_u \mathbf{t}^{(k)} = \mathbf{B}_h^T \mathbf{p}_h(\boldsymbol{\mu}^{(k)}), \text{ where: } \mathbf{B}_h^T = \begin{bmatrix} \mathbf{B}_{xD}^T \\ \mathbf{B}_{yD}^T \end{bmatrix}, \quad \mathbf{X}_u = \begin{bmatrix} \mathbf{A}_D & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_D \end{bmatrix}.$$

By first constructing the global saddle-point matrix $\mathbf{K}$ with unit viscosity ($\nu = 1$) and enforcing homogeneous Dirichlet boundary conditions, the required operators are extracted directly via matrix slicing. Slicing rows $[0, 2N_v)$ from $\mathbf{K}$ yields a rectangular matrix with the corresponding needed operators:

- the velocity block operator $\mathbf{X}_u = \mathbf{K}[0 : 2N_v, 0 : 2N_v] \in \mathbb{R}^{2N_v \times 2N_v}$,
- the discrete gradient block $\mathbf{B}_h^T = \mathbf{K}[0 : 2N_v, 2N_v : 2N_v + N_p] \in \mathbb{R}^{2N_v \times N_p}$.

Because both matrices are extracted from the exact same Dirichlet-corrected rows of $\mathbf{K}$, they naturally inherit identical boundary treatments. This approach directly reuses the existing Stokes assembly pipeline while keeping the underlying computations explicit and transparent.

Critically, $\mathbf{X}_u$ is assembled only once at $\nu = 1$, rather than at the viscosity described by parameter vector $\boldsymbol{\mu}^{(k)}$ of each snapshot. Since the discrete gradient matrix $\mathbf{B}_h^T$ depends solely on geometry and carries no viscosity dependence, computing the supremizer solutions $\mathbf{t} = \mathbf{X}_u^{-1} \mathbf{B}_h^T \mathbf{p}$ involves a single, parameter-independent matrix inverse $\mathbf{X}_u^{-1}$, applicable unchanged to every pressure snapshot regardless of which $\boldsymbol{\mu}^{(k)}$ produced it.

Collecting these linear solves over the full snapshot set yields the multi-column global system:

$$\mathbf{X}_u \mathbf{S}_t = \mathbf{B}_h^T \mathbf{S}_p.$$

In practice, rather than computing an explicit inverse, $\mathbf{X}_u$ is converted to CSC format and its sparse LU factorization is computed once. The complete multi-column right-hand side matrix $\mathbf{B}_h^T \mathbf{S}_p \in \mathbb{R}^{2N_v \times n_s}$ is then assembled in a single step, allowing the entire matrix of supremizer snapshots $\mathbf{S}_t$ to be resolved via back-substitution in one pass:

$$\mathbf{S}_t = \mathbf{X}_u^{-1} (\mathbf{B}_h^T \mathbf{S}_p) \in \mathbb{R}^{2N_v \times n_s}, \quad \mathbf{S}_t = \left[\mathbf{t}^{(1)} \mid \dots \mid \mathbf{t}^{(n_s)} \right] \in \mathbb{R}^{193\,634 \times 100}$$

Reusing the pre-factored LU operator across all n_s snapshot columns simultaneously drastically reduces the computational cost compared to solving snapshot vectors sequentially or refactorizing per snapshot.

9.3 Velocity Snapshot Homogenization via Boundary Lifting

To ensure that the reduced velocity basis functions belong to the homogeneous space X with $\mathbf{u}|_{\Gamma_D} = \mathbf{0}$, the inhomogeneous boundary data must be removed from the raw full-order velocity snapshots prior to POD.

Following the weak formulation decomposition introduced above (1), every total physical velocity field snapshot $\mathbf{u}(\mathbf{x}; \boldsymbol{\mu}^{(k)})$ is represented as:

$$\mathbf{u}(\mathbf{x}; \boldsymbol{\mu}^{(k)}) = \mathbf{u}_h(\mathbf{x}; \boldsymbol{\mu}^{(k)}) + \mathbf{r}_g(\mathbf{x}),$$

where $\mathbf{r}_g \in \mathbb{R}^{2N_v}$ is the discrete parameter-independent lifting vector defined earlier (3):

Given the raw velocity snapshot matrix $\mathbf{S}_u$, the homogenized snapshot matrix $\mathbf{S}_{u_h} \in \mathbb{R}^{2N_v \times n_s}$ is obtained by subtracting the lifting vector column-wise across all n_s snapshots:

$$\mathbf{S}_{u_h} = \mathbf{S}_u - \mathbf{r}_g \mathbf{1}_{n_s}^T \in \mathbb{R}^{2N_v \times n_s}, \quad (4)$$

where $\mathbf{1}_{n_s} \in \mathbb{R}^{n_s}$ is a vector of ones. Every column vector $\mathbf{u}_h^{(k)}$ in $\mathbf{S}_{u_h}$ strictly satisfies homogeneous Dirichlet boundary conditions ($\mathbf{u}_h^{(k)} \in X$), ensuring that all POD basis modes extracted from $\mathbf{S}_u$ naturally inherit homogeneous boundary values by construction.

10 Reduced Basis Construction

This section details the methodology for extracting a reduced basis from high-fidelity snapshots, capturing the system's core dynamics at a fraction of the full computational cost.

10.1 Proper Orthogonal Decomposition and Energy Truncation

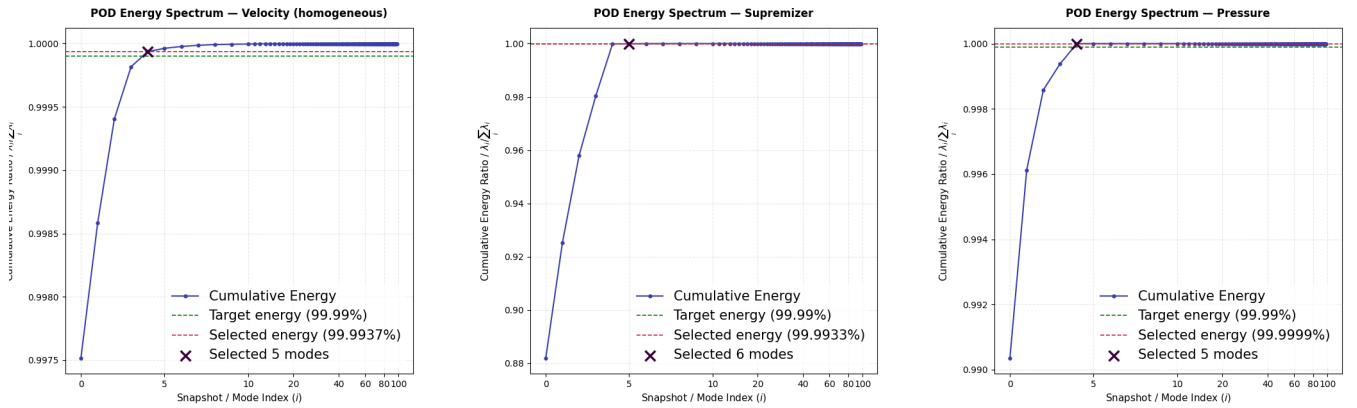


Figure 10: Cumulative POD energy spectra and mode selection thresholds for the velocity, supremizer, and pressure snapshot spaces.

10.2 Reduced Velocity and Pressure Spaces

11 Offline–Online Decomposition

11.1 Offline Phase: Reduced Operator Assembly

11.2 Online Phase: The Reduced Galerkin System

12 Numerical Results

- 12.1 Train/Test Partitioning of the Parameter Space
- 12.2 Reconstruction Accuracy: Seen vs. Unseen Parameters
- 12.3 Velocity and Pressure Field Comparison (FOM vs. ROM)
- 12.4 Derived Quantities of Interest
- 12.5 Computational Speedup

13 Appendix

13.1 Reverse Techniques: Einsum

First Einsum

```
A_local = np.zeros(shape=(Nt, 3, 3))
for i in range(3):
    for j in range(i, 3):

        grad_i = test_function_derivatives[i]
        grad_j = test_function_derivatives[j]

        val = np.einsum('i,nij,j->n',
                        grad_j, Q_mat, grad_i)

        val *= kinematic_viscosity / (2 * det_J)

        A_local[:, i, j] = val
        A_local[:, j, i] = val
```

$$\begin{array}{ccc}
 [\psi_1, \psi_2] & \begin{bmatrix} q_{11}^n & q_{12}^n \\ q_{21}^n & q_{22}^n \end{bmatrix} & [\psi_1, \psi_2] \\
 \psi_i & q_{ij}^n & \psi_j \\
 \sum_{i=2}^2 \sum_{j=2}^2 \psi_i q_{ij}^n \psi_j = A^n \in \mathbb{R}^n
 \end{array}$$

Second Einsum

```
A_local = np.einsum('mi,txy,nj->tmn',
                    test_function_derivatives, Q_mat, test_function_derivatives)
A_local *= (kinematic_viscosity / (2.0 * det_J))[:, None, None]
```

$$\begin{array}{ccc}
 \begin{bmatrix} [\psi_1^1, \psi_2^1] \\ [\psi_1^2, \psi_2^2] \\ [\psi_1^3, \psi_2^3] \end{bmatrix} & \begin{bmatrix} q_{11}^t & q_{12}^t \\ q_{21}^t & q_{22}^t \end{bmatrix} & \begin{bmatrix} [\psi_1^1, \psi_2^1] \\ [\psi_1^2, \psi_2^2] \\ [\psi_1^3, \psi_2^3] \end{bmatrix} \longrightarrow \begin{bmatrix} A_{11}^t & A_{12}^t & A_{13}^t \\ A_{21}^t & A_{22}^t & A_{23}^t \\ A_{31}^t & A_{32}^t & A_{33}^t \end{bmatrix} \\
 \psi_i^m & q_{xy}^t & \psi_j^n \qquad \qquad \qquad A_{mn}^t
 \end{array}$$

So by using indexation and sums we notice that the structure has not really changed since the first example 13.1:

$$\sum_{i=1}^2 \sum_{j=1}^2 \psi_i^m q_{ij}^t \psi_j^n = A_{mn}^t \in \mathbb{R}^{t \times m \times n}$$